instituto
superior de
engenharia
de lisboa

Licenciatura em Engenharia Informática e de Computadores

SEGURANÇA INFORMÁTICA

Trabalho 1

Relatório do Primeiro Trabalho
Semestre de Inverno de 2026/27

Gonçalo Amigo (50548), Inês Gil (52848)

Outubro de 2026

Índice

Introdução **1**

Exercício 1 – Confidencialidade e autenticidade de ficheiros **2**

1.1 Padding 2

1.2 Esquema de proteção 3

1.3 Recuperação do ficheiro original 4

Exercício 2 – Assinaturas digitais RSA com OpenSSL **5**

2.1 Geração do par de chaves de Alice 7

2.2 Assinatura e verificação das imagens 9

2.3 Robustez do esquema de assinatura 13

Exercício 3 – Hierarquia de certificados **20**

3.1 Criação da CA raiz, CA intermédia e certificado de servidor 20

3.2 Impressões digitais SHA-256 dos três certificados 23

3.3 Algoritmos de assinatura e relação com a chave do certificado 23

3.4 Quatro testes de validação da cadeia de certificados 25

Exercício 4 – Blockchain com a biblioteca JCA **28**

4.1 Experiência com 100 blocos e alvo 1024 29

4.2 Experiência com alvo 512: resultados e comparação 32

4.3 Alteração do quinto bloco e estimativa do trabalho necessário 34

Exercício 5 – Proteção de texto com JWE e a biblioteca JCA **35**

5.1 Alínea a) – Cifra e decifra autenticada AES-GCM 35

5.2 Alínea b) – Proteção da chave AES com RSA-OAEP 36

5.3 Alínea c) – Certificados X.509 e keystores PKCS#12 37

5.4 Alínea d) – Integração na aplicação JSON Web Encryption 38

Conclusão **42**

Exercício – Referências **44**

Lista de Figuras

1 Execução dos comandos de cifra AES-128-CBC, cálculo do HMAC-SHA256 e concatenação dos ficheiros, retirada do relatório original do grupo. 2

2 Comparação das dimensões de `alice.txt` e do criptograma `g`, utilizando `wc -c`. 2

3 Comando de decifra com a opção `-nopad`, que conserva os bytes do preenchimento para análise. 3

4 Visualização hexadecimal dos últimos 16 bytes do ficheiro decifrado, incluindo os seis bytes de padding. 3

5 Separação do criptograma e do HMAC, comparação das marcas, decifra e comparação do ficheiro recuperado com o original. 4

6 Camadas do algoritmo de assinatura S. 6

7 Percurso do algoritmo de verificação V e as duas barreiras. 7

8 Geração da chave privada RSA da Alice 8

9 Criação do ficheiro *alice.priv.pem*. 8

10 Terminal com `openssl version`, a geração das chaves e o resultado de `-text -noout`. 9

11 Comandos de assinatura e resultado de `ls -l`. 10

12 Duas verificações com `Verified OK`. 11

13 Verificação do caso de referência. 14

14 `cmp -l`, verificação falhada e hashes dos dois ficheiros. 16

15 Geração das chaves de Mallory, `diff` e as duas verificações. 18

16 Hierarquia de certificação construída pelo grupo (Exercício 3). 20

17 Geração da chave e do certificado da CA raiz (captura do grupo). 21

18 Geração do pedido e emissão do certificado da CA intermédia. 22

19 Geração do par RSA do servidor e emissão do certificado folha. 23

20 Impressões digitais SHA-256 de `root.pem`, `inter.pem` e `leaf.pem`. 23

21 Certificado raiz: algoritmo de assinatura RSA e chave pública RSA. 24

22 Certificado da CA intermédia: chave pública EC e assinatura da raiz RSA. . . . 25

23 Certificado do servidor: chave pública RSA e assinatura ECDSA da intermédia. . 25

24 Validação sem CA intermédia e com CA intermédia (testes 1 e 2). 26

25 Verificação do hostname correto e do hostname incorreto (testes 3 e 4). 26

26 Exercício 4: execução inicial da aplicação `Blockchain`, com criação de dois blocos e respetivos ficheiros, a partir da captura fornecida pelo grupo. 29

27 Exercício 4.1: primeira parte dos registos da experiência de 100 blocos, com alvo 1024. 30

28 Exercício 4.1: registos finais e médias obtidas pelo grupo com alvo 1024. 31
29 Exercício 4.2: primeira parte da experiência de 100 blocos com alvo 512. 32
30 Exercício 4.2: registos finais e médias da experiência com alvo 512. 33
31 Exercício 5(a): recuperação da mensagem AES-GCM e rejeição de um criptograma intencionalmente alterado. 36
32 Exercício 5(b): chave AES cifrada com RSA-OAEP e comparação positiva após a recuperação. 36
33 Exercício 5(c): validação do certificado de Alice e identificação da sua chave pública RSA. 37
34 Exercício 5(c): validação do certificado, recuperação da chave privada do PFX e confirmação da igualdade da chave AES. 37
35 Exercício 5(d): token JWE gerado e confirmação da existência das cinco partes previstas na serialização compacta. 39
36 Exercício 5(d): execução das operações de consola `enc` e `dec`, com recuperação da mensagem original. 39
37 Exercício 5(d): teste integrado de criação e decifra do JWE, com comparação positiva das mensagens. 39
38 Exercício 5(d): mensagem recuperada do token original e rejeição de um token com criptograma alterado. 40

Lista de Tabelas

1 Propriedades de segurança das funções de hash 6
2 As duas barreiras do algoritmo de verificação V 7
3 Dimensão dos ficheiros do exercício 2.2 10
4 O que Bob tem e o que não tem 10
5 Estrutura do conteúdo recuperado da assinatura 12
6 Propriedades do hash e cenários de ataque que impedem 13
7 Significado da saída de `cmp -l` 15
8 Quadro comparativo dos casos de verificação 19
9 Enquadramento das experiências 2.3.1 e 2.3.2 nas propriedades de segurança . . 19
10 Chave pública contida no certificado e algoritmo de assinatura. 24
11 Resultados observados nos testes de validação documentados pelo grupo. 26
12 Organização dos ficheiros de cada bloco na implementação apresentada. 28
13 Resultados observados pelo grupo para o alvo 1024. 30
14 Comparação dos resultados experimentais para os alvos 1024 e 512. 32

15 Estimativa da reconstrução da cadeia após alteração do quinto bloco. 34
16 Elementos da serialização compacta JWE utilizada na aplicação. 38

Introdução

A segurança da informação assume um papel essencial na proteção dos dados e das comunicações em sistemas informáticos. A utilização de mecanismos criptográficos permite responder a diferentes necessidades de segurança, nomeadamente a **confidencialidade**, a **integridade** e a **autenticidade** da informação. Para compreender estas propriedades, é importante não só conhecer os fundamentos dos algoritmos, mas também saber aplicá-los, interpretar os resultados obtidos e reconhecer as limitações de cada solução.

O presente trabalho, desenvolvido no âmbito da unidade curricular de Segurança Informática, tem como objetivo consolidar os conhecimentos de criptografia e de infraestruturas de chave pública através da realização de experiências práticas. Para esse efeito, são utilizadas duas ferramentas principais: o *OpenSSL*, para executar operações criptográficas e gerir certificados através da linha de comandos, e a *Java Cryptography Architecture* (JCA), para implementar e testar componentes criptográficos em Java.

A primeira parte aborda a proteção de ficheiros recorrendo a cifra simétrica e a códigos de autenticação de mensagem (MAC), a utilização de assinaturas digitais RSA com a função de resumo SHA-256 e a criação de uma hierarquia de certificação baseada em certificados digitais X.509. Estas experiências permitem observar a relação entre as operações realizadas, as chaves utilizadas e as propriedades de segurança que se pretendem garantir, incluindo a validação de assinaturas e de cadeias de certificação.

Na segunda parte, os conceitos são aplicados ao desenvolvimento de programas em Java. É estudada a construção e a verificação de uma cadeia de blocos e é implementada uma aplicação de consola para proteger mensagens no formato *JSON Web Encryption* (JWE). Nesta última aplicação, a cifra autenticada AES-GCM é combinada com a proteção de chaves por RSA-OAEP, utilizando certificados X.509 e *keystores* para obter o material criptográfico do destinatário.

Ao longo do relatório são apresentados os procedimentos seguidos, os comandos e os componentes de código utilizados, acompanhados das respetivas explicações e dos resultados experimentais. A análise dos testes permite relacionar a fundamentação teórica com o comportamento observado nas execuções, incluindo situações em que a verificação criptográfica é bem-sucedida e situações em que são detetadas alterações ou condições inválidas.

Exercício 1 – Confidencialidade e autenticidade de ficheiros

Esquema: `alice.txt` $\longrightarrow$ cifra $\longrightarrow g \longrightarrow$ HMAC$(g) \longrightarrow h$.

`result = g || h`

```
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/files_certificates-keys$
openssl enc -e -aes-128-cbc -in alice.txt -out g -iv 0ed425637bb85abc97908b7abde9980a -K 42c2771f8072158
02dff135054852848
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/files_certificates-keys$
openssl mac -digest SHA256 -macopt key:changeit -in g -out h HMAC
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/files_certificates-keys$
cat g h > result
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/files_certificates-keys$
ls
alice.txt  certificates-keys  g  h  imagem_10MB.png  imagem_1KB.png  result
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/files_certificates-keys$
```

Figura 1: Execução dos comandos de cifra AES-128-CBC, cálculo do HMAC-SHA256 e concatenação dos ficheiros, retirada do relatório original do grupo.

Explicação adicional dos comandos utilizados. Na primeira operação, `openssl enc` invoca a funcionalidade de cifra do OpenSSL. A opção `-e` seleciona a operação de cifra; `-aes-128-cbc` escolhe o AES com chave de 128 bits em modo CBC; `-in alice.txt` indica o ficheiro de entrada e `-out g` identifica o ficheiro que contém o criptograma. A opção `-iv` fornece o vetor de inicialização e `-K` especifica a chave simétrica diretamente em hexadecimal. Na execução documentada, o grupo utilizou a chave `42c2771f807215802dff135054852848`, construída com os números dos elementos do grupo, como solicitado no enunciado.

Na segunda operação, `openssl mac` produz um código de autenticação da mensagem. A opção `-digest SHA256` seleciona a função SHA-256, `-macopt key:changeit` define a chave utilizada no HMAC, `-in g` escolhe o criptograma como entrada, `-out h` indica o ficheiro de saída e `HMAC` identifica o mecanismo usado. Por último, `cat g h > result` concatena o criptograma `g` com a marca de autenticação `h`, por esta ordem, criando o ficheiro `result`. Esta descrição acompanha as operações presentes na captura e no enunciado do trabalho [1].

1.1 Padding

Mostra a quantidade de bytes entre o ficheiro original(alice.txt) e o ficheiro decifrado(g):

```
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/files_certificates-
keys$ wc -c alice.txt g
152138 alice.txt
152144 g
304282 total
```

Figura 2: Comparação das dimensões de `alice.txt` e do criptograma `g`, utilizando `wc -c`.

quantidade de bytes em alice.txt = 152 138 bytes

quantidade de bytes em g = 152 144 bytes

152144 - 152138 = 6 bytes de padding adicionados;

Explicação adicional da diferença de tamanhos. O AES-CBC processa blocos de 16 bytes. Como o ficheiro original tem 152 138 bytes, faltam 6 bytes para completar o último bloco. A cifra acrescentou automaticamente o preenchimento segundo PKCS#7: os seis bytes adicionados possuem todos o valor hexadecimal `06`. O número total de bytes do ficheiro cifrado ficou, assim, múltiplo de 16. Este preenchimento é efetuado pela operação de cifra AES-CBC e não pelo cálculo do HMAC.

podemos observar no seguinte comando que decifra o ficheiro g, e guarda o resultado com padding no ficheiro mensagem_original.txt

-nopad serve para manter o padding após a decifra

```
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/files_certificates-
keys$ openssl enc -d -aes-128-cbc -nopad -in g -out mensagem_original.txt -iv 0ed425637bb85abc9790
8b7abde9980a -K 42c2771f807215802dff135054852848
```

Figura 3: Comando de decifra com a opção `-nopad`, que conserva os bytes do preenchimento para análise.

Explicação adicional da opção `-nopad`. A opção `-d` solicita a decifra, enquanto `-nopad` impede que o OpenSSL retire automaticamente o preenchimento ao concluir a operação. Deste modo, o ficheiro `mensagem_original.txt` conserva os bytes de padding, que podem ser inspecionados diretamente. O vetor de inicialização e a chave usados na decifra são os mesmos utilizados na cifra.

mostra os ultimos 16 bytes do ficheiro decifrado:

```
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/files_certificates-keys$
tail -c 16 mensagem_original.txt | hexdump -C
00000000  20 54 48 45 20 45 4e 44  0d 0a 06 06 06 06 06 06  | THE END........|
00000010
```

Figura 4: Visualização hexadecimal dos últimos 16 bytes do ficheiro decifrado, incluindo os seis bytes de padding.

foi adicionado 6 bytes de padding, localizados no fim do ficheiro decifrado, por

0x06 06 06 06 06

Confirmação dos bytes observados. A captura mostra que o ficheiro termina com `06 06 06 06 06 06`, ou seja, **seis** ocorrências do valor hexadecimal `06`. Na linha anterior do texto original foram enumeradas apenas cinco ocorrências, por lapso de transcrição; o total de seis está confirmado tanto pelo output como pela diferença dos tamanhos. O comando `tail -c 16` seleciona os últimos 16 bytes e `hexdump -C` apresenta-os em hexadecimal e em representação textual.

1.2 Esquema de proteção

Os comandos implementam um esquema encrypt-then-MAC. Primeiro, alice.txt é cifrado com AES-128-CBC, produzindo g. Em seguida, é calculado um HMAC-SHA256 sobre o criptograma

g, produzindo h. Finalmente, g e h são concatenados em result. Assim, o MAC protege o texto cifrado, e não o texto original.

Seria MAC-then-encrypt se primeiro fosse calculado o HMAC(alice.txt), juntasse esse HMAC à mensagem e só depois fosse cifrado o conjunto.

Explicação adicional da classificação. A ordem das operações é determinante: como o HMAC é calculado *depois* da cifra e recebe o criptograma `g`, trata-se de *encrypt-then-MAC*. A cifra AES-CBC proporciona confidencialidade; o HMAC-SHA256 permite detetar modificações do criptograma desde que a chave do HMAC seja conhecida apenas pelas entidades autorizadas. Para recuperar o texto de forma segura, a marca deve ser verificada *antes* de se tentar decifrar `g`. O ficheiro `result` reúne os dois elementos necessários ao processo, pela ordem $g \| h$.

1.3 Recuperação do ficheiro original

`result` ⟶ separar `g` e `h` ⟶ verificar HMAC(g) ⟶ decifrar `g` ⟶ `alice\_recuperado.txt`

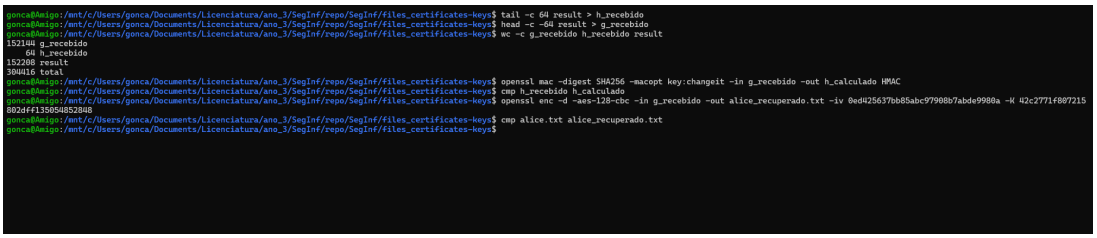


Figura 5: Separação do criptograma e do HMAC, comparação das marcas, decifra e comparação do ficheiro recuperado com o original.

Explicação adicional da sequência de recuperação. A sequência registada na captura começa por separar o ficheiro `result` em `g_recebido` e `h_recebido`. O comando `tail -c 64` obtém os 64 bytes finais, correspondentes ao HMAC-SHA256 guardado em *texto hexadecimal*; o comando `head -c -64` obtém os bytes anteriores, que constituem o criptograma. Na execução apresentada, os tamanhos foram 152 144 bytes para `g_recebido`, 64 bytes para `h_recebido` e 152 208 bytes para `result`, o que confirma a concatenação das duas partes.

Em seguida, o programa calcula novamente o HMAC-SHA256 sobre `g_recebido`, utilizando a mesma chave definida na proteção, e guarda o resultado em `h_calculado`. A comparação entre `h_recebido` e `h_calculado` verifica se as marcas coincidem. No teste apresentado, `cmp` não indicou diferenças, pelo que foi possível prosseguir com a decifra. A verificação antes da decifra é essencial para evitar processar um criptograma cuja autenticidade não tenha sido confirmada.

Finalmente, a operação `openssl enc -d` utiliza AES-128-CBC, a chave e o IV originais para recuperar `alice_recuperado.txt`. Desta vez não é utilizada a opção `-nopad`, pelo que o preenchimento é removido automaticamente durante a decifra. O último comando `cmp` compara o ficheiro recuperado com `alice.txt`; a ausência de diferenças no output apresentado é compatível com uma recuperação correta do conteúdo original.

Exercício 2 – Assinaturas digitais RSA com OpenSSL

Objetivo do exercício 2

Este exercício consiste em usar o OpenSSL para gerar chaves RSA e para assinar e verificar mensagens, seguindo o esquema de assinatura digital (G, S, V) estudado nas aulas [3]. O exercício divide-se em três partes:

- **2.1** – gerar o par de chaves RSA de 2048 bits de Alice (K_s^{Alice}, chave privada; K_v^{Alice}, chave pública).
- **2.2** – assinar duas imagens (1 KB e 10 MB) com SHA-256, verificar as assinaturas como Bob (apenas com a chave pública) e responder à pergunta teórica sobre a propriedade de segurança garantida por uma função de hash adequada.
- **2.3** – testar a robustez do esquema: **2.3.1** verificar a assinatura original sobre uma mensagem alterada num único carácter; **2.3.2** verificar, com a chave pública de Alice, uma assinatura feita por Mallory, comparar com verificações bem-sucedidas e enquadrar os resultados nas propriedades de segurança da assinatura digital.

Cada comando é apresentado com a explicação de cada uma das suas partes, o resultado observado e a respetiva interpretação. Seguiram-se os exemplos de comandos do repositório da disciplina [4].

Convenções. Alice é quem assina, Bob é quem verifica e Mallory é a atacante. As chaves são geradas aleatoriamente, por isso os valores das chaves e das assinaturas mudam em cada execução. Os resultados dos hashes dependem apenas do conteúdo dos ficheiros e são, por isso, reprodutíveis.

Fundamentação teórica

Esquema de assinatura digital (G, S, V)

Um esquema de assinatura digital é um trio de algoritmos [3]:

- G gera o par de chaves (k_s, k_v).
- $S(k_s)(m)$ assina a mensagem m com a chave privada k_s e produz a assinatura s.
- $V(k_v)(s, m)$ verifica a assinatura com a chave pública k_v e devolve verdadeiro ou falso. É um algoritmo determinístico.

A **assinatura tem dimensão fixa** (por exemplo, 2048 bits), enquanto a **mensagem** é uma sequência de bytes de dimensão variável.

Arquitetura interna dos algoritmos S e V

O algoritmo S tem três camadas:

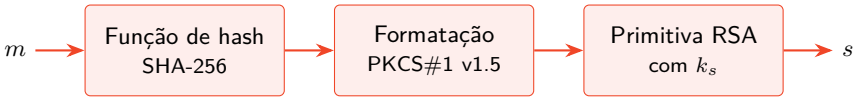


Figura 6: Camadas do algoritmo de assinatura S.

1. **Função de hash:** calcula $h = \text{SHA-256}(m)$, com 32 bytes, independentemente do tamanho de m.
2. **Formatação (padding):** h é embrulhado num bloco com o tamanho do módulo (256 bytes para 2048 bits), com a estrutura `00 01 | FF .. FF | 00 | DigestInfo`. O DigestInfo tem 51 bytes: um prefixo de 19 bytes que identifica o SHA-256, seguido dos 32 bytes do hash.
3. **Primitiva RSA com k_s:** o bloco é elevado ao expoente privado, módulo n, o que produz a assinatura s, com 256 bytes.

O algoritmo V faz o percurso inverso: recalcula o hash da mensagem recebida, aplica a primitiva RSA com k_v (bloco $= s^e \bmod n$), remove a formatação (Deformat) e compara os dois resumos. Assinar **não** é "cifrar com a chave privada": a primitiva matemática é semelhante, mas as operações e os objetivos são diferentes.

Propriedades de segurança das funções de hash

Uma função de hash $H\colon \{0,1\}^* \to \{0,1\}^n$ aceita entradas de qualquer tamanho e produz uma saída de tamanho fixo (no SHA-256, $n = 256$ bits, ou seja, 32 bytes). As propriedades de segurança relevantes [2] são:

Tabela 1: Propriedades de segurança das funções de hash

Propriedade	Definição
Segunda pré-imagem	dado x (e $H(x)$), é computacionalmente difícil obter $x' \neq x$ com $H(x') = H(x)$
Colisão	é computacionalmente difícil obter qualquer par (x, x'), com $x' \neq x$, tal que $H(x) = H(x')$

Segurança do esquema de assinatura

- **Correção:** $V(k_v)(S(k_s)(m), m) =$ verdadeiro. Uma assinatura gerada por S é sempre aceite por V sobre a mesma mensagem.
- **Segurança:** sem conhecer k_s, é computacionalmente infazível obter uma **falsificação seletiva** (dada m, encontrar s tal que $V(k_v)(s, m) =$ verdadeiro) ou uma **falsificação existencial** (encontrar algum par (m, s) válido).

Terminologia usada neste relatório: as duas barreiras do algoritmo V

Para explicar onde e porquê uma verificação falha, divide-se o algoritmo V em duas verificações sucessivas. Esta designação é própria deste relatório (não é terminologia dos slides):

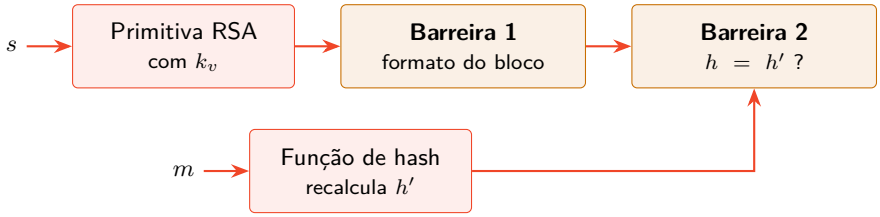


Figura 7: Percurso do algoritmo de verificação V e as duas barreiras.

Tabela 2: As duas barreiras do algoritmo de verificação V

Barreira	Pergunta	Se falhar
Barreira 1 (formato)	Ao aplicar k_v à assinatura, sai um bloco bem formado (`00 01 FF..FF 00 | DigestInfo`)?	a chave não corresponde à que assinou
Barreira 2 (hash)	O hash contido no bloco é igual ao hash recalculado da mensagem?	a mensagem não é a que foi assinada

O resultado final é sempre falso quando qualquer uma das barreiras falha. O que muda é a causa.

2.1 Geração do par de chaves de Alice

2.1.1 Verificação da versão do OpenSSL

```
openssl version
```

Explicação adicional do comando. O comando consulta a versão da implementação criptográfica disponível no computador. Esta verificação documenta o ambiente de execução e permite interpretar diferenças entre mensagens apresentadas pelo LibreSSL e por outras versões do OpenSSL.

Resultado: `LibreSSL 3.3.6`. Os comandos usados neste exercício (`genpkey`, `pkey`, `dgst`, `pkeyutl`) são suportados por esta versão.

2.1.2 Geração da chave privada

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out alice_priv.pem
```

Explicação adicional do comando. `genpkey` gera a chave privada RSA de Alice; `-algorithm RSA` seleciona o algoritmo, `-pkeyopt rsa_keygen_bits:2048` estabelece o tamanho do módulo e `-out` identifica o ficheiro PEM de destino. A chave privada deve permanecer protegida: é ela que permite produzir assinaturas em nome de Alice.

Parte	Significado
`openssl`	o programa
`genpkey`	"generate private key": gera uma chave privada
`-algorithm RSA`	tipo de chave
`-pkeyopt rsa_keygen_bits:2048`	opção específica do RSA: tamanho do módulo, 2048 bits
`-out alice_priv.pem`	ficheiro de saída



Figura 8: Geração da chave privada RSA da Alice

Os valores da chave (módulo n, primos p e q, expoente privado d) são escolhidos aleatoriamente pelo OpenSSL. O que se define é o algoritmo e o tamanho. Escolher manualmente o valor seria inseguro (valores com padrão são previsíveis) e matematicamente inválido (p e q têm de ser primos com certas propriedades).

2.1.3 Extração da chave pública

```
openssl pkey -in alice_priv.pem -pubout -out alice_pub.pem
```

Explicação adicional do comando. `pkey` lê a chave privada indicada em `-in`; a opção `-pubout` extrai apenas a parte pública e `-out` grava-a em `alice_pub.pem`. O ficheiro público pode ser entregue a Bob para verificar assinaturas, sem divulgar a chave privada de Alice.

Foi criado o ficheiro `alice_pub.pem`, que começa por `-----BEGIN PUBLIC KEY-----` e termina em `-----END PUBLIC KEY-----`. É bastante mais pequeno do que o da chave privada, porque só contém o módulo n e o expoente público e.



Figura 9: Criação do ficheiro *alice.priv.pem.*

2.1.4 Verificação das chaves

```
openssl pkey -in alice_pub.pem -pubin -text -noout
```

Explicação adicional do comando. `-pubin` informa que a entrada é uma chave pública; `-text` apresenta os parâmetros RSA e `-noout` evita imprimir novamente a representação PEM. O módulo de 2048 bits confirma o tamanho configurado e o expoente público é apresentado em formato legível.

Parte	Significado
`-pubin`	indica que o ficheiro de entrada é uma chave **pública**
`-text`	mostra os valores em formato legível
`-noout`	não repete o PEM no fim

```
inesgil@MacBook-Air-de-Ines exercicio2 % openssl pkey -in alice_pub.pem -pubin -text -noout
RSA Public-Key: (2048 bit)
Modulus:
    00:be:61:83:25:cf:e0:5a:b6:bd:49:88:19:55:89:
    b1:47:3c:38:dd:dd:bb:3b:26:e1:48:9b:c9:4b:96:
    0f:7d:2e:03:87:91:0a:04:e6:dc:a8:64:0e:a8:9f:
    26:b0:e5:6c:14:6e:d9:ce:5c:78:90:14:df:eb:40:
    a3:93:4e:89:66:99:5e:9b:55:30:2e:95:e9:7a:2e:
    4d:22:b4:80:b3:b2:c5:db:28:07:be:a5:da:17:12:
    98:37:78:b9:da:09:57:d0:76:9d:95:d0:55:a0:89:
    14:3d:fa:20:ff:7e:8e:42:cc:96:36:5e:f8:62:08:
    13:86:8a:89:e7:45:c0:0c:e8:34:d0:74:48:28:f4:
    fa:5f:2b:1e:cc:14:0c:dc:15:7c:95:62:64:41:ea:
    68:6a:33:fa:bf:39:21:a2:e1:3a:09:b4:12:a5:0a:
    c6:de:f0:be:59:07:53:34:1a:99:aa:a3:46:04:78:
    9b:ef:cf:ee:51:dc:0c:31:ca:c2:41:16:4a:96:b8:
    bc:7a:2f:2f:c6:ea:cc:d9:82:b8:47:42:9d:31:0c:
    cd:4b:b6:4d:d5:fb:62:5d:2d:d0:58:2a:2e:d1:a6:
    ee:4d:7a:c2:8e:48:2f:fb:9c:04:59:89:75:2e:c3:
    d4:69:d6:90:f5:69:fd:d4:71:40:30:74:78:f1:cd:
    77:35
Exponent: 65537 (0x10001)
inesgil@MacBook-Air-de-Ines exercicio2 %
```

Figura 10: Terminal com `openssl version`, a geração das chaves e o resultado de `-text -noout`.

2.2 Assinatura e verificação das imagens

2.2.1 Assinatura das duas imagens (algoritmo S)

```
openssl dgst -sha256 -sign alice_priv.pem -out imagem_1KB.sig imagem_1KB.png
openssl dgst -sha256 -sign alice_priv.pem -out imagem_10MB.sig imagem_10MB.png
```

Explicação adicional do comando. Primeiro comando: calcula SHA-256 da imagem pequena e produz a assinatura RSA com a chave privada de Alice, guardando-a em `imagem_1KB.sig`. **Segundo comando:** realiza a mesma operação para a imagem grande, guardando a assinatura em `imagem_10MB.sig`. Embora as mensagens tenham dimensões muito diferentes, ambas as assinaturas usam o mesmo tamanho de chave RSA.

Parte	Significado
`dgst`	comando para calcular resumos (digests) e assinaturas
`-sha256`	função de hash da primeira camada
`-sign alice_priv.pem`	assina com a chave privada de Alice (k_s)
`-out imagem_1KB.sig`	onde guardar a assinatura s
`imagem_1KB.png`	a mensagem m a assinar (vai sempre no fim do comando)

O OpenSSL executa as três camadas num único comando (hash, formatação PKCS#1 v1.5 e primitiva RSA com k_s). Os comandos não produzem qualquer output no ecrã, o que indica que

correram sem erro.

2.2.2 Dimensão dos ficheiros

```
ls -l imagem_1KB.png imagem_10MB.png imagem_1KB.sig imagem_10MB.sig
```

Explicação adicional do comando. `ls -l` apresenta a dimensão dos quatro ficheiros: duas imagens originais e duas assinaturas. Esta comparação permite verificar experimentalmente que a dimensão da assinatura RSA é determinada pelo módulo RSA e não pelo tamanho do ficheiro assinado.

Tabela 3: Dimensão dos ficheiros do exercício 2.2

Ficheiro	Tamanho
`imagem_1KB.png`	6 794 bytes (≈ 6,8 KB)
`imagem_10MB.png`	10 491 530 bytes (≈ 10,5 MB)
`imagem_1KB.sig`	**256 bytes**
`imagem_10MB.sig`	**256 bytes**

As duas imagens têm tamanhos muito diferentes, mas as duas assinaturas têm **exatamente 256 bytes (2048 bits)**: o tamanho da assinatura depende da chave (do módulo n) e não da mensagem. Isto confirma a dimensão fixa da assinatura referida na teoria.

```
inesgil@MacBook-Air-de-Ines exercicio2 % openssl dgst -sha256 -sign alice_priv.pem -out imagem_1KB.sig imagem_1KB.png
inesgil@MacBook-Air-de-Ines exercicio2 % openssl dgst -sha256 -sign alice_priv.pem -out imagem_10MB.sig imagem_10MB.png
inesgil@MacBook-Air-de-Ines exercicio2 % ls -l imagem_1KB.png imagem_10MB.png imagem_1KB.sig imagem_10MB.sig
-rwx------@ 1 inesgil  staff  10491530 Sep 19 16:20 imagem_10MB.png
-rw-r--r--@ 1 inesgil  staff       256 Oct 12 14:27 imagem_10MB.sig
-rwx------@ 1 inesgil  staff      6794 Sep 19 16:20 imagem_1KB.png
-rw-r--r--@ 1 inesgil  staff       256 Oct 12 14:14 imagem_1KB.sig
```

Figura 11: Comandos de assinatura e resultado de `ls -l`.

2.2.3 Verificação como Bob (algoritmo V)

O que Bob tem e o que não tem:

Tabela 4: O que Bob tem e o que não tem

Bob tem	Bob não tem
`alice_pub.pem` (k_v)	`alice_priv.pem` (k_s)
a mensagem recebida (`imagem_1KB.png`)	
a assinatura (`imagem_1KB.sig`)	

```
openssl dgst -sha256 -verify alice_pub.pem -signature imagem_1KB.sig imagem_1KB.png
openssl dgst -sha256 -verify alice_pub.pem -signature imagem_10MB.sig imagem_10MB.png
```

Explicação adicional do comando. Primeiro comando: Bob utiliza a chave pública de Alice e a assinatura da imagem pequena para verificar essa mesma imagem. **Segundo comando:**

repete a verificação para a imagem grande. Em ambos os casos, `-sha256` garante que é calculado o resumo com a mesma função de hash utilizada por Alice na assinatura.

Parte	Significado
`dgst -sha256`	Bob tem de usar a **mesma** função de hash que Alice usou
`-verify alice_pub.pem`	verifica com a chave **pública** (k_v)
`-signature imagem_1KB.sig`	a assinatura s que veio com a mensagem
`imagem_1KB.png`	a mensagem m recebida

O que o OpenSSL faz por dentro:

1. Recalcula h' = SHA-256 da imagem recebida.
2. Aplica a chave pública à assinatura (bloco = $s^e \bmod n$), o que desfaz a camada de assinatura.
3. Remove a formatação (Deformat) e extrai o hash h e o identificador do SHA-256.
4. Compara h com h'. Se forem iguais, a verificação é bem-sucedida.

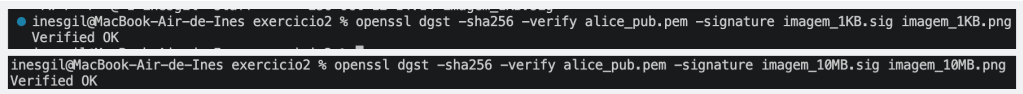


Figura 12: Duas verificações com `Verified OK`.

2.2.4 Conteúdo da assinatura

A chave pública consegue abrir a assinatura e mostrar o hash que está lá dentro:

```
openssl pkeyutl -verifyrecover -pubin -inkey alice_pub.pem -in imagem_1KB.sig | xxd -p -c 64
```

Explicação adicional do comando. `pkeyutl -verifyrecover` recupera a estrutura codificada na assinatura RSA PKCS#1 v1.5 por aplicação da chave pública. O operador `|` envia os bytes recuperados para `xxd`; `-p -c 64` apresenta-os em hexadecimal. O resultado permite identificar o *DigestInfo* que contém a identificação do SHA-256 e o resumo da mensagem.

Parte	Significado
`pkeyutl -verifyrecover`	aplica a operação pública e devolve o que a assinatura contém
`-pubin -inkey alice_pub.pem`	usa a chave pública
`-in imagem_1KB.sig`	a assinatura
`xxd -p -c 64`	mostra os bytes em hexadecimal, numa só linha

Resultado (51 bytes):

```
3031300d060960864801650304020105000420 8
    ↪ c4055c610c5722a5a5f4b5e6fd07128d1f942a6bb2798e65ebaeb0f80790a5a
```

(O espaço foi acrescentado apenas para separar as duas partes. No terminal aparece tudo se-

guido.)

Tabela 5: Estrutura do conteúdo recuperado da assinatura

Parte	Bytes	Significado
`3031300d060960864801650304020105000420`	19	prefixo ASN.1 que identifica o SHA-256
`8c4055c6 … 790a5a`	32	o **hash da imagem** que Alice assinou

Isto mostra que a assinatura contém o **hash** da mensagem e não a mensagem. O procedimento repete-se para a imagem de 10 MB.

2.2.5 Hashes das imagens

```
openssl dgst -sha256 imagem_1KB.png imagem_10MB.png
```

Explicação adicional do comando. `dgst -sha256` calcula diretamente os resumos das duas imagens sem utilizar as assinaturas. A comparação dos valores com os recuperados das assinaturas confirma que foi usado o hash do conteúdo de cada imagem na operação de assinatura.

```
SHA256(imagem_1KB.png)=  8c4055c610c5722a5a5f4b5e6fd07128d1f942a6bb2798e65ebaeb0f80790a5a
SHA256(imagem_10MB.png)= 58fa8a24ac7cd2fc57d5cbaec7ea8364f4f6d0ed9f4a3fd30d6df59a82ac949d
```

Cada hash tem 64 caracteres hexadecimais (2 por byte), ou seja, 32 bytes = 256 bits, para duas imagens de tamanhos muito diferentes: é a definição $H\colon \{0,1\}^* \to \{0,1\}^n$. O hash de `imagem_1KB.png` coincide com o recuperado de dentro da assinatura na secção 2.2.4.

2.2.6 Controlo negativo: emparelhamento errado

Para mostrar que cada assinatura está ligada à sua mensagem, verificou-se a assinatura da imagem pequena com a imagem grande:

```
openssl dgst -sha256 -verify alice_pub.pem -signature imagem_1KB.sig imagem_10MB.png
```

Explicação adicional do comando. Neste controlo negativo, mantém-se a assinatura da imagem pequena mas fornece-se a imagem grande para verificação. Como a assinatura está associada ao resumo da primeira imagem, o resumo recalculado sobre a segunda é diferente e a verificação deve falhar.

Resultado: `Verification Failure`. O hash recalculado da imagem de 10 MB é diferente do que está dentro da assinatura da imagem de 1 KB.

2.2.7 Resposta à pergunta teórica

Que propriedade de segurança é garantida ao ser usada uma função de hash adequada?

O que Bob verifica na realidade. Bob compara **hashes**: o que está dentro da assinatura

com o que ele próprio recalcula. Nunca compara a mensagem inteira. A assinatura RSA liga a chave de Alice a $H(m)$ e não diretamente a m.

Qual é o risco. Se um atacante conseguisse arranjar uma mensagem diferente m' com $H(m') = H(m)$, a mesma assinatura s serviria para as duas, e Bob aceitaria m' como se fosse de Alice.

Propriedades que impedem isto:

Tabela 6: Propriedades do hash e cenários de ataque que impedem

Propriedade	Definição	Cenário que impede
Segunda pré-imagem	dado x (e $H(x)$), é computacionalmente difícil obter $x' \neq x$ com $H(x') = H(x)$	o atacante tem uma assinatura válida de m e quer outra mensagem que a reutilize
Colisão	é computacionalmente difícil obter qualquer par (x, x'), $x' \neq x$, com $H(x) = H(x')$	o atacante prepara um par (m_1, m_2) com o mesmo hash, convence Alice a assinar m_1 e apresenta m_2

Na segunda pré-imagem o atacante parte de uma mensagem **dada**, enquanto na colisão **escolhe os dois**, o que é mais fácil. Por isso, a resistência a colisões é a mais exigente das duas. Os slides da disciplina distinguem os dois casos: na segunda pré-imagem, dados x_2 e h_2, encontrar x_4; na colisão, encontrar (x_3, x_5) ou (x_2, x_4).

O que o SHA-256 oferece. O SHA-256 tem 2^{256} imagens possíveis, o que torna a procura de uma segunda pré-imagem computacionalmente inviável.

O que a experiência mostra e não mostra. Mostra que uma assinatura de 256 bytes cobre ficheiros de 6,8 KB e de 10,5 MB, porque o RSA só trabalha sobre os 32 bytes do hash. Não prova a resistência a colisões, que é uma propriedade **assumida** do SHA-256.

Resposta

Na verificação, Bob recalcula o hash SHA-256 da mensagem recebida e compara-o com o hash recuperado da assinatura através da chave pública de Alice. Como a assinatura RSA é aplicada ao hash e não à mensagem completa, as duas imagens, com 6,8 KB e 10,5 MB, produzem assinaturas com o mesmo tamanho (256 bytes, o tamanho do módulo de 2048 bits), e a verificação deu `Verified OK` em ambos os casos.
A segurança desta construção depende da função de hash: se fosse fácil encontrar uma mensagem $m' \neq m$ com o mesmo hash, a assinatura de m seria válida também para m'. Por isso, a função de hash tem de ser resistente à segunda pré-imagem e a colisões. Com o SHA-256, é computacionalmente difícil encontrar tais mensagens, pelo que assinar o hash equivale a assinar a mensagem, o que garante a **integridade** do conteúdo assinado.

2.3 Robustez do esquema de assinatura

Assinatura de `alice.txt` e caso de referência

```
openssl dgst -sha256 -sign alice_priv.pem -out alice.sig alice.txt
```

```
openssl dgst -sha256 -verify alice_pub.pem -signature alice.sig alice.txt
```

Explicação adicional do comando. Primeiro comando: Alice assina o texto original, produzindo `alice.sig`. **Segundo comando:** confirma que a assinatura é válida quando é verificada com a chave pública de Alice e com o mesmo ficheiro `alice.txt`. Esta verificação positiva serve de referência para o teste de alteração da mensagem.

Parte	Significado
`-sha256`	função de hash da primeira camada da assinatura
`-sign alice_priv.pem`	assina com a chave privada k_s (sem esta opção, o comando só calcularia o hash)
`-out alice.sig`	ficheiro onde fica a assinatura s
`-verify alice_pub.pem`	verifica com a chave pública k_v (a única que Bob tem)
`-signature alice.sig`	a assinatura s recebida
`alice.txt`	a mensagem m, que vai sempre no fim do comando

```
inesgil@MacBook-Air-de-Ines exercicio2 % openssl dgst -sha256 -verify alice_pub.pem -signature alice.sig alice.txt
Verified OK
```

Figura 13: Verificação do caso de referência.

Percurso do algoritmo V: hash $h' = \text{SHA-256}(\texttt{alice.txt})$; primitiva bloco $= s^e \bmod n$ (desfaz a camada de assinatura, porque e e d são inversos); barreira 1 (o bloco tem a estrutura `00 01 FF..FF 00 | DigestInfo`?); barreira 2 ($h = h'$?).

Conteúdo da assinatura.

```
openssl pkeyutl -verifyrecover -pubin -inkey alice_pub.pem -in alice.sig | xxd -p -c 64
```

Explicação adicional do comando. A opção `-verifyrecover` permite observar a estrutura interna da assinatura válida do texto original, incluindo a identificação do algoritmo de hash e o resumo SHA-256. Esse resumo é o valor que será comparado com o hash da mensagem na verificação.

Os primeiros 19 bytes (`3031300d060960864801650304020105000420`) identificam o SHA-256. Os últimos 32 bytes, `d67c71` … `3d94`, são o **hash de `alice.txt`**, igual ao calculado diretamente:

```
SHA256(alice.txt)= d67c718cc4c14a864a5ffd9bc5555a2c0d74c54071f085e27f53b1bf84bc3d94
```

2.3.1 Mensagem alterada

Passo 1 – cópia da mensagem.

```
cp alice.txt alice_mod.txt
```

Explicação adicional do comando. `cp` cria uma cópia do ficheiro original para que seja possível modificar apenas a cópia, preservando o conteúdo assinado em `alice.txt`. O ficheiro

`alice_mod.txt` é utilizado no teste em que se tenta reutilizar a assinatura original.

Passo 2 – alteração de um único carácter. Abriu-se `alice_mod.txt` num editor de texto (VS Code) e substituiu-se um único carácter: na linha do título, "CHAPTER I" passou a "CHAPTER A". A alteração foi uma **substituição** (e não uma remoção), por isso o tamanho do ficheiro manteve-se.

Passo 3 – confirmação de que só um byte mudou.

```
cmp -l alice.txt alice_mod.txt
```

Explicação adicional do comando. `cmp -l` compara os dois ficheiros byte a byte e apresenta, quando existem diferenças, a posição e os valores dos bytes alterados. Deste modo confirma-se que a experiência modifica efetivamente a mensagem, sem substituir a assinatura.

Resultado, uma só linha:

```
191 111 101
```

Tabela 7: Significado da saída de `cmp -l`

`Valor`	**Significado**
`191`	o byte número 191 do ficheiro (contagem a partir de 1)
`111`	valor **octal** desse byte em `alice.txt`, a letra `I`
`101`	valor **octal** desse byte em `alice_mod.txt`, a letra `A`

Passo 4 – verificação da assinatura original sobre a mensagem alterada.

```
openssl dgst -sha256 -verify alice_pub.pem -signature alice.sig alice_mod.txt
```

Explicação adicional do comando. A verificação utiliza a assinatura válida de `alice.txt`, mas fornece a mensagem modificada `alice_mod.txt`. Como o SHA-256 da mensagem alterada não corresponde ao resumo associado à assinatura, o resultado esperado é `Verification Failure`.

Resultado: `Verification Failure`.

Passo 5 – justificação com os hashes.

```
openssl dgst -sha256 alice.txt alice_mod.txt
```

Explicação adicional do comando. O cálculo independente de SHA-256 para `alice.txt` e `alice_mod.txt` permite comparar diretamente os resumos antes e depois da alteração. Um único byte diferente é suficiente para produzir, neste caso, hashes distintos e explicar o insucesso da verificação anterior.

```
SHA256(alice.txt)=     d67c718cc4c14a864a5ffd9bc5555a2c0d74c54071f085e27f53b1bf84bc3d94
SHA256(alice_mod.txt)= 98d77daa9bf048fa2e6cd8370d23247f9f9a6d159ffd7daab56d26ac36dda7e1
```

Uma diferença de um único byte em 152 138 fez com que **61 dos 64 caracteres** hexadecimais do hash fossem diferentes.

Interpretação

Etapa do algoritmo V	**Resultado**
Hash recalculado $h' = \text{SHA-256}(\texttt{alice_mod.txt})$	`98d77d … a7e1`
Aplicação de k_v de Alice a `alice.sig`	bloco bem formado (a assinatura é autêntica)
Barreira 1 (formato)	passa
Hash contido na assinatura h	`d67c71 … 3d94` (o da mensagem original)
Barreira 2 ($h = h'$)	**falha**: `d67c71 … 3d94` $\neq$ `98d77d … a7e1`

A verificação da assinatura original sobre `alice_mod.txt` falhou porque a assinatura contém o hash SHA-256 de `alice.txt` (`d67c71 … 3d94`) e o hash recalculado sobre a mensagem alterada (`98d77d … a7e1`) é diferente. A assinatura não mudou, o que mudou foi a mensagem. Alterar um único byte (de 152 138) bastou para que o hash mudasse quase por completo. Isto mostra que a assinatura garante a **integridade** da mensagem: reaproveitá-la para outra mensagem exigiria encontrar uma segunda pré-imagem do hash, o que é computacionalmente difícil. Também não é viável criar uma nova assinatura para a mensagem alterada, porque isso exigiria k_s de Alice.

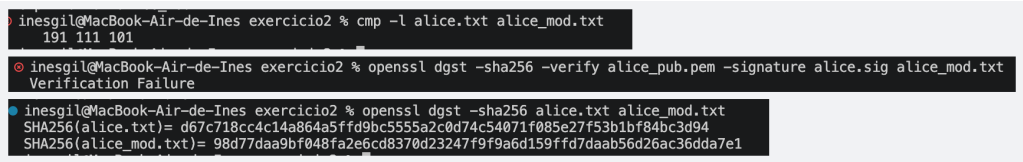


Figura 14: `cmp -l`, verificação falhada e hashes dos dois ficheiros.

2.3.2 Assinatura de Mallory

Neste cenário, Mallory não tem a chave privada de Alice. Assina `alice.txt` com a **sua própria chave** e tenta que Bob aceite essa assinatura como se fosse de Alice.

Passo 1 – par de chaves de Mallory.

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out mallory_priv.pem
openssl pkey -in mallory_priv.pem -pubout -out mallory_pub.pem
```

Explicação adicional do comando. Primeiro comando: gera a chave privada RSA de Mallory com módulo de 2048 bits. **Segundo comando:** extrai a chave pública correspondente. São chaves próprias de Mallory e diferentes das de Alice, condição necessária para testar uma tentativa de fazer passar a assinatura de Mallory por uma assinatura de Alice.

São os mesmos comandos da alínea 2.1, com outros nomes de ficheiro (explicação das opções na secção 2.1). Usaram-se também **2048 bits**, para que a única coisa que muda entre os dois casos seja **quem assina** e não a força da chave.

Passo 2 – confirmar que as chaves são diferentes.

```
ls mallory_*
diff alice_pub.pem mallory_pub.pem | head -3
```

Explicação adicional do comando. `ls mallory_*` confirma os ficheiros de chaves criados para Mallory. A operação `diff` compara o conteúdo das chaves públicas de Alice e de Mallory; o encadeamento `| head -3` limita a visualização às primeiras linhas de diferença. O objetivo é assegurar que se trata de pares de chaves distintos.

O `ls` mostra `mallory_priv.pem` e `mallory_pub.pem`. O `diff` devolveu diferenças logo a partir da linha 2 (`2,8c2,8`), o que prova que a chave pública de Mallory **não** é a de Alice. Isto é essencial: se por engano se usasse a chave de Alice como "chave de Mallory", a assinatura seria de Alice e a verificação passaria. A experiência só faz sentido com duas identidades diferentes, cada uma com o seu par (k_s, k_v).

Passo 3 – Mallory assina `alice.txt`.

```
openssl dgst -sha256 -sign mallory_priv.pem -out alice_mallory.sig alice.txt
```

Explicação adicional do comando. Mallory utiliza `-sign mallory_priv.pem` para assinar o mesmo ficheiro `alice.txt`, mas com a sua própria chave privada. O resultado em `alice_mallory.sig` é uma assinatura de Mallory; a sua validade depende de ser verificada com a chave pública correspondente.

Parte	Significado
`dgst -sha256`	hash SHA-256, a primeira camada da assinatura
`-sign mallory_priv.pem`	assina com a chave privada de Mallory (k_s de Mallory)
`-out alice_mallory.sig`	onde guardar a assinatura (nome escolhido por nós)
`alice.txt`	a mensagem, exatamente a mesma que Alice assinou

O hash é o mesmo da assinatura de Alice (`d67c71` … `3d94`), porque a mensagem é a mesma. O que muda é a última camada, que usa o expoente privado e o módulo de Mallory. Assinar com a própria chave é legítimo: a assinatura de Mallory é válida **para a chave pública de Mallory**. O ataque consiste em apresentá-la como sendo de Alice.

Passo 4 – verificação com a chave pública de Alice.

```
openssl dgst -sha256 -verify alice_pub.pem -signature alice_mallory.sig alice.txt
```

Explicação adicional do comando. Este comando tenta verificar a assinatura criada por Mallory com a chave pública de Alice. Como as chaves não pertencem ao mesmo par RSA, a assinatura não pode ser aceite como sendo de Alice e a verificação deverá falhar.

Parte	Significado
`-verify alice_pub.pem`	Bob usa a chave pública que julga ser de Alice
`-signature alice_mallory.sig`	a assinatura recebida (feita por Mallory)
`alice.txt`	a mensagem, que não foi alterada

Resultado: `Verification Failure`.

Interpretação

Porque falha.

1. Bob recalcula o hash: $h' = \text{SHA-256}(\texttt{alice.txt}) = \texttt{d67c71}\ldots\texttt{3d94}$. Está correto, a mensagem não mudou.
2. Aplica a chave pública de Alice à assinatura: bloco $= s^{e_A} \bmod n_A$. Mas a assinatura foi produzida com $s = b^{d_M} \bmod n_M$; para a desfazer seria preciso e_M e n_M, que são de Mallory. Com os valores de Alice, o resultado é um número de 2048 bits **sem qualquer relação** com o bloco original.
3. Deformat: o bloco legítimo tem de começar por `00 01 FF FF .. FF 00` e depois o DigestInfo. Um número obtido ao acaso não cumpre esta estrutura (a probabilidade é desprezável), pelo que a verificação falha logo na **barreira 1**.

Passo 5 – comparação com verificações bem-sucedidas.

5.1 Caso de referência (assinatura de Alice com a chave de Alice):

```
openssl dgst -sha256 -verify alice_pub.pem -signature alice.sig alice.txt
```

Explicação adicional do comando. Este controlo positivo volta a verificar a assinatura original de Alice com a chave pública correspondente e com a mensagem não alterada. O resultado esperado é `Verified OK`, permitindo comparar um caso legítimo com a tentativa de falsificação.

Resultado: `Verified OK`.

5.2 Assinatura de Mallory com a chave de Mallory (prova que a assinatura dela não está estragada, apenas não é de Alice):

```
openssl dgst -sha256 -verify mallory_pub.pem -signature alice_mallory.sig alice.txt
```

Explicação adicional do comando. Este segundo controlo positivo verifica a assinatura de Mallory utilizando a própria chave pública de Mallory. Um resultado `Verified OK` mostra que a assinatura foi produzida corretamente; a falha no teste anterior decorreu da utilização da chave pública de Alice e não de um erro na assinatura de Mallory.

Resultado: `Verified OK`.

O ficheiro é o mesmo nos dois casos e a assinatura de Mallory é válida. O único fator que decide o resultado é **com que chave pública se verifica**.



Figura 15: Geração das chaves de Mallory, `diff` e as duas verificações.

Quadro comparativo

Tabela 8: Quadro comparativo dos casos de verificação

Caso	Mensagem	Assinatura	Chave	Resultado	Onde falha
Referência	`alice.txt`	de Alice	k_v de Alice	**Verified OK**	–
2.3.1	`alice_mod.txt`	de Alice	k_v de Alice	**Failure**	barreira 2: hash diferente
2.3.2	`alice.txt`	de Mallory	k_v de Alice	**Failure**	barreira 1: chave não corresponde
2.3.2 (controlo)	`alice.txt`	de Mallory	k_v de Mallory	**Verified OK**	–

Nas duas falhas, o veredicto é o mesmo (falso), mas a causa é diferente: em 2.3.1 a **mensagem** mudou, em 2.3.2 a **chave** não corresponde a quem assinou.

Enquadramento nas propriedades de segurança da assinatura digital

- **Correção.** A verificação de referência e o controlo de Mallory dão `Verified OK`: quando a assinatura e a chave correspondem, $V(k_v)(S(k_s)(m), m) = \text{verdadeiro}$.
- **Segurança contra falsificação.** Sem conhecer k_s, é computacionalmente infazível obter uma falsificação seletiva ou existencial. As duas experiências simulam tentativas de falsificação, que falharam.

Tabela 9: Enquadramento das experiências 2.3.1 e 2.3.2 nas propriedades de segurança

Exp.	Ataque simulado	Porque não funciona	O que ilustra
2.3.1	reaproveitar a assinatura de m para uma mensagem m' escolhida pelo atacante	$H(m') \neq H(m)$; só funcionaria com uma segunda pré-imagem do hash, o que é computacionalmente difícil	**integridade**
2.3.2	Mallory assina m com a sua chave e apresenta o resultado como sendo de Alice	sem k_s de Alice não é possível produzir uma assinatura que k_v de Alice aceite; a de Mallory só é válida para k_v de Mallory	**autenticidade** da origem

Exercício 3 – Hierarquia de certificados

Neste exercício, foi construída uma hierarquia de certificação com três níveis: uma CA raiz, uma CA intermédia e um certificado de servidor, também designado certificado folha. Foi utilizado o número **50548**, correspondente ao menor número de aluno do grupo. A raiz emitiu o certificado da intermédia, que, por sua vez, emitiu o certificado do servidor `www.aluno50548.isel.pt`.

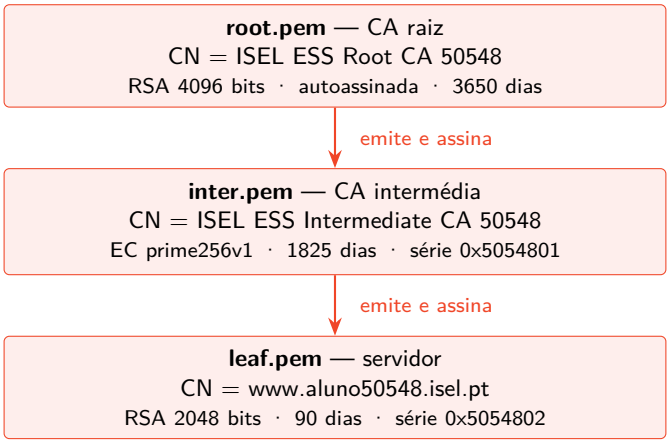


Figura 16: Hierarquia de certificação construída pelo grupo (Exercício 3).

3.1 Criação da CA raiz, CA intermédia e certificado de servidor

Ficheiros de configuração

Os ficheiros de configuração definem as extensões X.509 a incluir nos certificados. Segue-se o conteúdo presente no repositório do grupo, preservando a sintaxe do OpenSSL.

Configuração da CA raiz — `root.cnf`

```
[ req ]
distinguished_name = dn
x509_extensions = ext
[ dn ]
[ ext ]
basicConstraints=critical,CA:TRUE
keyUsage=critical,keyCertSign,cRLSign
subjectKeyIdentifier=hash
```

O campo `basicConstraints` identifica o certificado como pertencente a uma autoridade certificadora. A extensão `keyUsage` autoriza a assinatura de certificados e de listas de revogação, enquanto `subjectKeyIdentifier` facilita a identificação da chave pública do certificado. O uso de `x509_extensions = ext` indica a secção de extensões utilizada ao produzir o certificado autoassinado.

Configuração da CA intermédia — `inter.cnf`

```
[ ext ]
```

```
basicConstraints=critical,CA:TRUE,pathlen:0
keyUsage=critical,keyCertSign,cRLSign
nameConstraints=critical,permitted;DNS:.isel.pt
subjectKeyIdentifier=hash
authorityKeyIdentifier=keyid:always
```

A CA intermédia também é identificada como CA, mas a restrição `pathlen:0` impede que sejam acrescentadas outras CAs intermédias abaixo dela na cadeia. A extensão `nameConstraints` limita os nomes DNS permitidos a `.isel.pt`. O `authorityKeyIdentifier` permite relacionar este certificado com a autoridade que o emitiu.

Configuração do certificado folha — `leaf.cnf`

```
[ ext ]
basicConstraints=critical,CA:FALSE
keyUsage=critical,digitalSignature,keyEncipherment
extendedKeyUsage=serverAuth
subjectAltName=DNS:www.aluno50548.isel.pt
subjectKeyIdentifier=hash
authorityKeyIdentifier=keyid:always
```

O certificado de servidor é identificado como entidade final (`CA:FALSE`). As extensões `keyUsage` e `extendedKeyUsage` especificam os usos admitidos para a chave e a autenticação de servidor TLS; `subjectAltName` regista o nome DNS que será confirmado nos testes de hostname.

Geração da CA raiz

Foi gerado um par de chaves RSA de 4096 bits para a CA raiz, ficando a chave privada armazenada em `root.key`. De seguida, foi criado o certificado autoassinado `root.pem`, com validade de 3650 dias e assinatura RSA com SHA-256. A identificação da raiz foi definida através de `-subj`, utilizando o número 50548. As extensões de `root.cnf` identificam a entidade como CA e permitem a utilização da sua chave para assinar certificados e listas de revogação.

```
NUM=50548
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -out root.key
openssl req -new -x509 -key root.key -sha256 -days 3650 \
  -out root.pem -subj "/C=PT/O=ISEL/CN=ISEL ESS Root CA $NUM" \
  -config root.cnf
```

Explicação: `genpkey` gera a chave RSA; `req -new -x509` cria um certificado autoassinado; `-sha256` identifica o hash utilizado na assinatura, `-days` define o período de validade, `-subj` identifica o titular e `-config` seleciona o ficheiro de configuração.

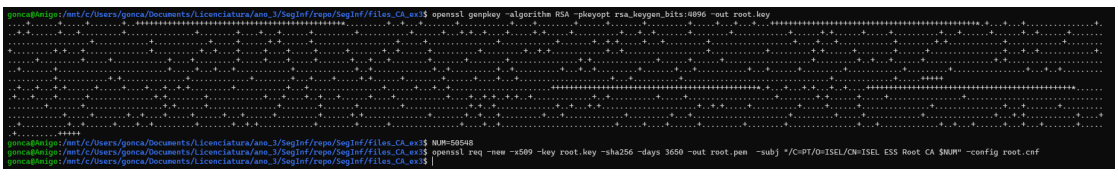


Figura 17: Geração da chave e do certificado da CA raiz (captura do grupo).

Geração da CA intermédia

Foi gerado um par de chaves EC com a curva `prime256v1`, ficando a chave privada armazenada em `inter.key`. A partir desta chave, foi criado o pedido de certificado `inter.csr`, contendo a identificação e a chave pública da intermédia. A CA raiz emitiu o certificado `inter.pem`, assinando-o com `root.key` e atribuindo-lhe o número de série `0x5054801` e uma validade de 1825 dias.

Foram aplicadas as extensões de `inter.cnf`, selecionando a secção `[ext]` com `-extensions ext`. Estas permitem à intermédia emitir certificados, restringem os nomes DNS subordinados aos subdomínios de `.isel.pt` e, através de `pathlen:0`, limitam a cadeia para que não admita outras CAs intermédias abaixo desta.

```
openssl genpkey -algorithm EC \
  -pkeyopt ec_paramgen_curve:prime256v1 -out inter.key
openssl req -new -key inter.key -out inter.csr \
  -subj "/C=PT/O=ISEL/CN=ISEL ESS Intermediate CA $NUM"
openssl x509 -req -in inter.csr -CA root.pem -CAkey root.key \
  -set_serial 0x${NUM}01 -days 1825 -sha256 \
  -extfile inter.cnf -extensions ext -out inter.pem
```

Explicação: os parâmetros de `genpkey` geram uma chave de curva elíptica; o ficheiro `.csr` contém o pedido de emissão; `-CA` e `-CAkey` indicam o certificado e a chave privada da entidade emissora, e `-set_serial` define o número de série. O comando `x509 -req` produz o certificado assinado pela raiz.

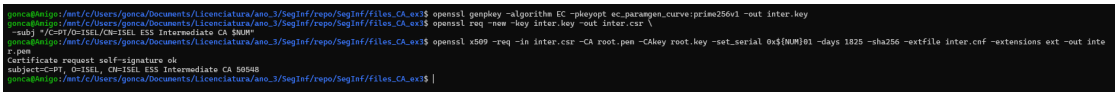


Figura 18: Geração do pedido e emissão do certificado da CA intermédia.

Geração do certificado de servidor

Foi gerado um par de chaves RSA de 2048 bits para o servidor, ficando a chave privada armazenada em `leaf.key`. De seguida, foi criado o pedido de certificado `leaf.csr`, associado ao nome `www.aluno50548.isel.pt`. A CA intermédia emitiu o certificado `leaf.pem`, utilizando a sua chave privada `inter.key`, com o número de série `0x5054802` e validade de 90 dias.

Foram aplicadas as extensões de `leaf.cnf`, selecionando a secção `[ext]`. Estas identificam o certificado como entidade final (`CA:FALSE`), permitem a autenticação de servidor TLS e incluem o nome DNS do servidor em `subjectAltName`.

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out leaf.key
openssl req -new -key leaf.key -out leaf.csr \
  -subj "/C=PT/O=ISEL/CN=www.aluno$NUM.isel.pt"
openssl x509 -req -in leaf.csr -CA inter.pem -CAkey inter.key \
  -set_serial 0x${NUM}02 -days 90 -sha256 \
  -extfile leaf.cnf -extensions ext -out leaf.pem
```

Explicação: a chave do servidor é distinta da chave da intermédia. O certificado final é emitido pela intermédia, a qual utiliza a sua chave privada EC para o assinar. O domínio configurado será utilizado no teste de validação de nome.



Figura 19: Geração do par RSA do servidor e emissão do certificado folha.

3.2 Impressões digitais SHA-256 dos três certificados

Foram calculadas as impressões digitais SHA-256 dos certificados da raiz, da intermédia e do servidor. Cada impressão digital corresponde a um resumo de 256 bits do certificado, apresentado como 64 algarismos hexadecimais separados por dois-pontos. Estes valores permitem identificar e comparar o conteúdo dos certificados.

```
openssl x509 -in root.pem  -noout -fingerprint -sha256
openssl x509 -in inter.pem -noout -fingerprint -sha256
openssl x509 -in leaf.pem  -noout -fingerprint -sha256
```

Explicação: o parâmetro `-fingerprint` calcula a impressão digital do certificado e `-sha256` seleciona a função de hash. Os valores obtidos são específicos dos certificados gerados, não das configurações OpenSSL.

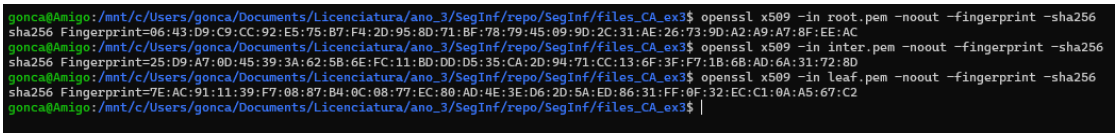


Figura 20: Impressões digitais SHA-256 de `root.pem`, `inter.pem` e `leaf.pem`.

3.3 Algoritmos de assinatura e relação com a chave do certificado

Para consultar o campo *Signature Algorithm*, o algoritmo da chave pública e as extensões de cada certificado, foram utilizados os comandos:

```
openssl x509 -in root.pem  -text -noout
openssl x509 -in inter.pem -text -noout
openssl x509 -in leaf.pem  -text -noout
```

Tabela 10: Chave pública contida no certificado e algoritmo de assinatura.

Certificado	Chave pública contida	*Signature Algorithm*	Assinado por
`root.pem`	RSA, 4096 bits	`sha256WithRSAEncryption`	CA raiz (autoassinado)
`inter.pem`	EC, curva `prime256v1`	`sha256WithRSAEncryption`	CA raiz (RSA)
`leaf.pem`	RSA, 2048 bits	`ecdsa-with-SHA256`	CA intermédia (EC)

O certificado raiz apresenta o algoritmo de assinatura `sha256WithRSAEncryption` e contém uma chave pública RSA de 4096 bits. Sendo autoassinado, a assinatura foi produzida com a chave privada da própria raiz. Os campos *Issuer* e *Subject* identificam a mesma entidade.

O certificado da intermédia contém uma chave pública EC de 256 bits, associada à curva `prime256v1`. Contudo, o seu algoritmo de assinatura é `sha256WithRSAEncryption`, uma vez que foi emitido e assinado pela CA raiz, cuja chave privada é RSA.

O certificado do servidor contém uma chave pública RSA de 2048 bits, mas apresenta o algoritmo de assinatura `ecdsa-with-SHA256`. Esta diferença deve-se ao facto de o certificado ter sido assinado pela CA intermédia, cuja chave privada é EC. Assim, **o algoritmo da assinatura depende da chave do emissor, enquanto a chave pública contida no certificado pertence ao titular**.

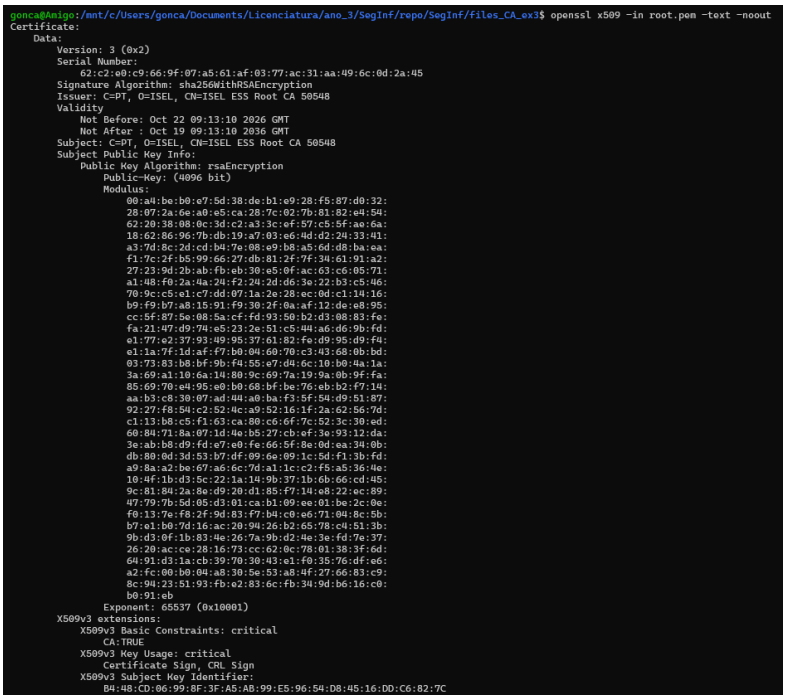


Figura 21: Certificado raiz: algoritmo de assinatura RSA e chave pública RSA.

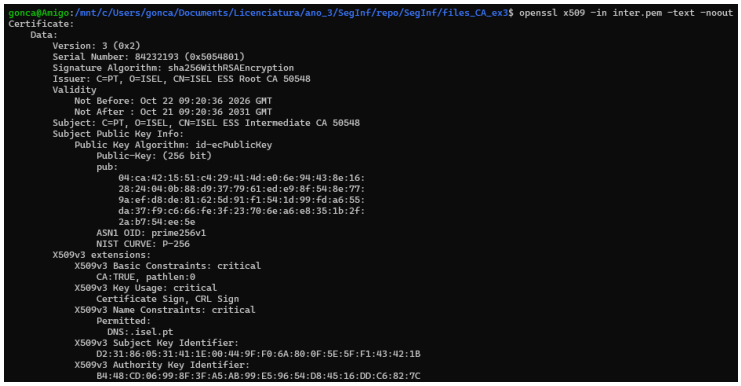


Figura 22: Certificado da CA intermédia: chave pública EC e assinatura da raiz RSA.

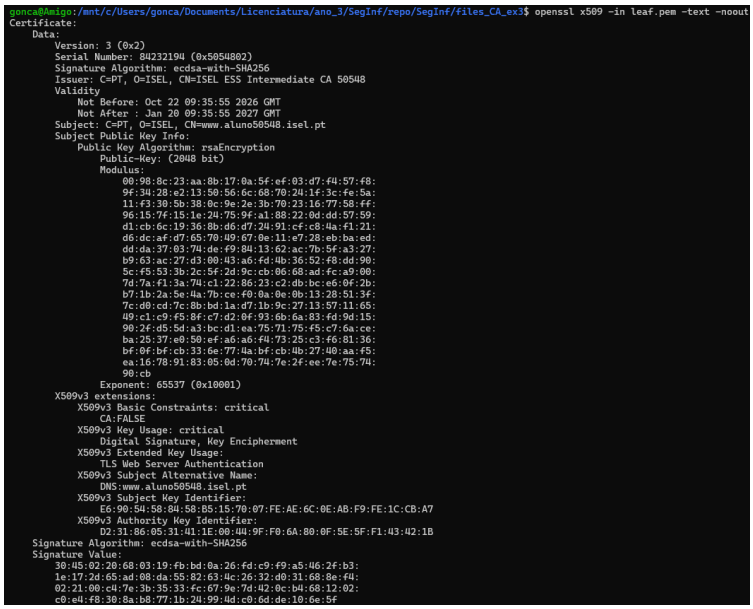


Figura 23: Certificado do servidor: chave pública RSA e assinatura ECDSA da intermédia.

3.4 Quatro testes de validação da cadeia de certificados

A validação foi efetuada com `openssl verify`, utilizando `root.pem` como âncora de confiança e `inter.pem` quando era necessário fornecer a CA intermédia. Nos testes de nome foi utilizada a opção `-verify_hostname`.

```
# Teste 1: sem fornecer o certificado da CA intermedia
openssl verify -CAfile root.pem leaf.pem

# Teste 2: fornecendo a CA intermedia
openssl verify -CAfile root.pem -untrusted inter.pem leaf.pem

# Teste 3: hostname presente no certificado
openssl verify -CAfile root.pem -untrusted inter.pem \
  -verify_hostname www.aluno50548.isel.pt leaf.pem
```

```
# Teste 4: hostname diferente do certificado
openssl verify -CAfile root.pem -untrusted inter.pem \
  -verify_hostname www.aluno50548.example.com leaf.pem
```

Tabela 11: Resultados observados nos testes de validação documentados pelo grupo.

Teste	Resultado apresentado	Erro	Interpretação
Sem intermédia	`unable to get local issuer certificate`	20	Falta o certificado da entidade emissora, pelo que não se constrói a cadeia até à raiz.
Com intermédia	`leaf.pem: OK`	–	A CA intermédia permite construir e validar o caminho até à raiz.
Nome `.isel.pt`	`leaf.pem: OK`	–	O nome solicitado corresponde ao `subjectAltName` do servidor.
Nome `.example.com`	`hostname mismatch`	62	O nome solicitado não corresponde ao nome DNS do certificado.

Para o primeiro teste, sem a intermédia, a validação falhou com o erro 20, `unable to get local issuer certificate`. Como o certificado do servidor foi emitido pela CA intermédia e esta não foi fornecida, o OpenSSL não conseguiu construir o caminho de certificação até à raiz de confiança.

Para o segundo teste, com a intermédia, a validação terminou com `leaf.pem: OK`. Ao fornecer `inter.pem` através de `-untrusted`, foi possível construir e validar a cadeia servidor → CA intermédia → CA raiz, sendo esta última a âncora de confiança indicada em `-CAfile`.

Para o terceiro teste, com o nome `.isel.pt`, a validação terminou com `leaf.pem: OK`, pois a cadeia é válida e o nome solicitado, `www.aluno50548.isel.pt`, corresponde ao nome DNS presente na extensão `subjectAltName` do certificado.

Para o quarto teste, com o nome `.example.com`, a validação falhou com o erro 62, `hostname mismatch`. O nome solicitado, `www.aluno50548.example.com`, não corresponde ao nome DNS identificado no certificado do servidor.

```
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/files_CA_ex3$ openssl verify -CAfile root.pem leaf.pem
C=PT, O=ISEL, CN=www.aluno50548.isel.pt
error 20 at 0 depth lookup: unable to get local issuer certificate
error leaf.pem: verification failed
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/files_CA_ex3$ openssl verify -CAfile root.pem -untrusted inter.pem leaf.pem
leaf.pem: OK
```

Figura 24: Validação sem CA intermédia e com CA intermédia (testes 1 e 2).

```
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/files_CA_ex3$ openssl verify -CAfile root.pem -untrusted inter.pem  -verify_hostname www.aluno$NUM.isel.pt leaf.pem
leaf.pem: OK
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/files_CA_ex3$ openssl verify -CAfile root.pem -untrusted inter.pem \
 -verify_hostname www.aluno$NUM.example.com leaf.pem
C=PT, O=ISEL, CN=www.aluno50548.isel.pt
error 62 at 0 depth lookup: hostname mismatch
error leaf.pem: verification failed
```

Figura 25: Verificação do hostname correto e do hostname incorreto (testes 3 e 4).

Conclusões do exercício 3

Foi construída uma hierarquia PKI de três níveis com uma raiz RSA autoassinada, uma CA intermédia com chave pública EC e um certificado de servidor RSA. Os ficheiros de configuração permitiram estabelecer restrições de utilização e de nome, e os testes demonstraram a importância da cadeia de confiança e da correspondência do nome DNS. Concluiu-se também que o algoritmo de assinatura de um certificado é determinado pela chave do seu emissor, e não necessariamente pelo algoritmo da chave pública do seu titular.

Exercício 4 – Blockchain com a biblioteca JCA

Objetivo e descrição da implementação

Neste exercício foi desenvolvida uma aplicação Java, denominada `Blockchain`, utilizando a classe `MessageDigest` da *Java Cryptography Architecture* (JCA) para calcular valores de SHA-256. A aplicação recebe uma mensagem como argumento, acrescenta um bloco ao fim da cadeia e guarda os ficheiros correspondentes na diretoria de execução. A experiência permite observar como os valores de *hash* ligam os blocos e como a exigência de uma condição sobre o resultado introduz um mecanismo simplificado de *proof of work* (prova de trabalho).

Cada bloco, de índice N, é representado por três ficheiros: `N.txt`, que contém a mensagem em UTF-8; `N.nce`, que regista o *nonce* encontrado; e `N.hsh`, que regista o valor H16 desse bloco. A numeração começa em 1. Ao inserir um novo bloco, é considerado o *hash* do bloco anterior; no primeiro bloco, utiliza-se o valor zero.

O H16 é obtido a partir dos primeiros 16 bits do SHA-256 aplicado à concatenação da mensagem do bloco, do *nonce* e do H16 anterior. De acordo com a descrição da implementação fornecida pelo grupo, para o cálculo da função de *hash* a mensagem é codificada em UTF-8, o *nonce* em 8 bytes e o H16 anterior em 2 bytes, com os valores numéricos em *big-endian*. O resultado é interpretado como um inteiro sem sinal entre 0 e 65535. Esta representação de entrada não deve ser confundida com a representação textual dos resultados guardados nos ficheiros, tal como ilustrado nos exemplos do terminal.

Tabela 12: Organização dos ficheiros de cada bloco na implementação apresentada.

Ficheiro	Conteúdo e finalidade
`N.txt`	Mensagem textual do bloco, em UTF-8.
`N.nce`	*Nonce* obtido durante a procura de uma solução para a prova de trabalho.
`N.hsh`	H16 do bloco, que permite verificar a sua ligação ao bloco anterior.

A pesquisa começa com *nonce* igual a zero, que é incrementado até se obter um H16 estritamente inferior ao valor alvo. Na configuração inicial, esse limite é 1024. Após encontrar um *nonce* válido, o programa escreve os três ficheiros e apresenta no terminal o índice, o H16, o número de tentativas e o tempo de execução. O tempo interno indicado corresponde ao intervalo desde a entrada no método `main` até à conclusão da escrita dos ficheiros.

```
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/blockchain_ex4$ javac --release 21
 -encoding UTF-8 Blockchain.java
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/blockchain_ex4$ mkdir -p dados
cd dados
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/blockchain_ex4/dados$ java -cp ..
Blockchain "Primeira mensagem do grupo 03"
Bloco criado: 1
Ficheiros: 1.txt, 1.nce, 1.hsh
Hash anterior: 0
Alvo: 1024
Nonce: 102
H16: 431 (0x01AF)
Tentativas: 103
Tempo de execucao: 81.380 ms
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/blockchain_ex4/dados$ java -cp ..
Blockchain "Segunda mensagem do grupo 03"
Bloco criado: 2
Ficheiros: 2.txt, 2.nce, 2.hsh
Hash anterior: 431
Alvo: 1024
Nonce: 56
H16: 737 (0x02E1)
Tentativas: 57
Tempo de execucao: 55.111 ms
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/blockchain_ex4/dados$ ls -l
total 0
-rwxrwxrwx 1 gonca gonca  3 Oct 22 16:45 1.hsh
-rwxrwxrwx 1 gonca gonca  3 Oct 22 16:45 1.nce
-rwxrwxrwx 1 gonca gonca 29 Oct 22 16:45 1.txt
-rwxrwxrwx 1 gonca gonca  3 Oct 22 16:45 2.hsh
-rwxrwxrwx 1 gonca gonca  2 Oct 22 16:45 2.nce
-rwxrwxrwx 1 gonca gonca 28 Oct 22 16:45 2.txt
```

Figura 26: Exercício 4: execução inicial da aplicação `Blockchain`, com criação de dois blocos e respetivos ficheiros, a partir da captura fornecida pelo grupo.

Na captura, o primeiro bloco apresenta H16 igual a 431 e 103 tentativas; o segundo apresenta H16 igual a 737 e 57 tentativas. Estes dois exemplos demonstram a procura de um *nonce* até satisfazer a condição H16 < 1024, mas não substituem as experiências com 100 blocos descritas nas alíneas seguintes.

4.1 Experiência com 100 blocos e alvo 1024

Foi realizada uma experiência partindo de uma cadeia vazia, com inserção de 100 blocos e valor alvo 1024. Para cada bloco, utilizou-se uma mensagem de 64 caracteres hexadecimais obtida a partir de 32 bytes aleatórios gerados com `SecureRandom`. O ensaio foi automatizado com o programa `Experiencia41`, que iniciou uma execução independente de `Blockchain` por inserção e guardou os resultados individuais num ficheiro CSV.

Para cada bloco foram registados o *nonce*, o H16, o número de tentativas, o tempo interno da aplicação e o tempo total do processo. Este último inclui o arranque da JVM e foi utilizado para representar o tempo por execução considerado no enunciado. O ensaio foi realizado em Ubuntu/WSL com Java 21.0.12.1.

Tabela 13: Resultados observados pelo grupo para o alvo 1024.

Indicador	Resultado
Número de blocos	100
Valor alvo	1024
Total de tentativas	6771
Média de tentativas por bloco	67,710
Tempo médio interno da aplicação	127,754 ms
Tempo médio total por processo	207,469 ms
Ambiente de execução	Ubuntu/WSL; Java 21.0.12.1

```
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/blockchain_ex4$ javac --release 21
 -encoding UTF-8 Blockchain.java Experiencia41.java
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/blockchain_ex4$ java Experiencia41
Resultados: /mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/blockchain_ex4/resultados_4_1/exec
ucao_20261022_175904_156
Insercoes: 100 | Alvo: 1024
Bloco 001/100 | tentativas: 146 | aplicacao: 43.239 ms | processo: 145.732 ms
Bloco 002/100 | tentativas: 11 | aplicacao: 102.973 ms | processo: 186.443 ms
Bloco 003/100 | tentativas: 33 | aplicacao: 53.091 ms | processo: 133.514 ms
Bloco 004/100 | tentativas: 11 | aplicacao: 61.601 ms | processo: 211.011 ms
Bloco 005/100 | tentativas: 145 | aplicacao: 59.721 ms | processo: 131.250 ms
Bloco 006/100 | tentativas: 117 | aplicacao: 55.180 ms | processo: 128.416 ms
Bloco 007/100 | tentativas: 15 | aplicacao: 53.453 ms | processo: 133.087 ms
Bloco 008/100 | tentativas: 23 | aplicacao: 76.496 ms | processo: 172.856 ms
Bloco 009/100 | tentativas: 102 | aplicacao: 67.234 ms | processo: 158.996 ms
Bloco 010/100 | tentativas: 49 | aplicacao: 75.759 ms | processo: 176.785 ms
Bloco 011/100 | tentativas: 27 | aplicacao: 60.364 ms | processo: 133.311 ms
Bloco 012/100 | tentativas: 14 | aplicacao: 73.120 ms | processo: 167.179 ms
Bloco 013/100 | tentativas: 26 | aplicacao: 74.534 ms | processo: 161.666 ms
Bloco 014/100 | tentativas: 124 | aplicacao: 72.242 ms | processo: 154.924 ms
Bloco 015/100 | tentativas: 5 | aplicacao: 77.524 ms | processo: 205.855 ms
Bloco 016/100 | tentativas: 247 | aplicacao: 89.869 ms | processo: 168.720 ms
Bloco 017/100 | tentativas: 139 | aplicacao: 76.049 ms | processo: 166.892 ms
Bloco 018/100 | tentativas: 174 | aplicacao: 73.985 ms | processo: 155.826 ms
Bloco 019/100 | tentativas: 6 | aplicacao: 70.232 ms | processo: 138.072 ms
Bloco 020/100 | tentativas: 15 | aplicacao: 66.482 ms | processo: 131.485 ms
Bloco 021/100 | tentativas: 14 | aplicacao: 69.942 ms | processo: 146.115 ms
Bloco 022/100 | tentativas: 77 | aplicacao: 102.923 ms | processo: 229.962 ms
Bloco 023/100 | tentativas: 137 | aplicacao: 92.902 ms | processo: 164.680 ms
Bloco 024/100 | tentativas: 33 | aplicacao: 76.027 ms | processo: 152.877 ms
Bloco 025/100 | tentativas: 38 | aplicacao: 75.899 ms | processo: 137.855 ms
Bloco 026/100 | tentativas: 160 | aplicacao: 78.115 ms | processo: 147.677 ms
Bloco 027/100 | tentativas: 73 | aplicacao: 83.308 ms | processo: 155.742 ms
Bloco 028/100 | tentativas: 19 | aplicacao: 73.670 ms | processo: 197.383 ms
Bloco 029/100 | tentativas: 31 | aplicacao: 96.986 ms | processo: 179.260 ms
Bloco 030/100 | tentativas: 154 | aplicacao: 90.454 ms | processo: 160.367 ms
Bloco 031/100 | tentativas: 146 | aplicacao: 81.253 ms | processo: 155.610 ms
Bloco 032/100 | tentativas: 142 | aplicacao: 84.841 ms | processo: 170.256 ms
Bloco 033/100 | tentativas: 16 | aplicacao: 80.487 ms | processo: 175.099 ms
Bloco 034/100 | tentativas: 194 | aplicacao: 96.492 ms | processo: 175.958 ms
Bloco 035/100 | tentativas: 89 | aplicacao: 111.608 ms | processo: 195.049 ms
Bloco 036/100 | tentativas: 79 | aplicacao: 110.902 ms | processo: 185.779 ms
Bloco 037/100 | tentativas: 5 | aplicacao: 91.113 ms | processo: 165.253 ms
Bloco 038/100 | tentativas: 31 | aplicacao: 94.226 ms | processo: 165.569 ms
Bloco 039/100 | tentativas: 66 | aplicacao: 95.527 ms | processo: 159.459 ms
Bloco 040/100 | tentativas: 4 | aplicacao: 96.864 ms | processo: 175.072 ms
Bloco 041/100 | tentativas: 10 | aplicacao: 94.881 ms | processo: 173.537 ms
Bloco 042/100 | tentativas: 15 | aplicacao: 102.980 ms | processo: 192.763 ms
Bloco 043/100 | tentativas: 25 | aplicacao: 99.601 ms | processo: 169.555 ms
Bloco 044/100 | tentativas: 48 | aplicacao: 105.753 ms | processo: 211.146 ms
Bloco 045/100 | tentativas: 37 | aplicacao: 107.430 ms | processo: 173.166 ms
Bloco 046/100 | tentativas: 110 | aplicacao: 135.119 ms | processo: 206.914 ms
Bloco 047/100 | tentativas: 54 | aplicacao: 121.573 ms | processo: 190.579 ms
Bloco 048/100 | tentativas: 9 | aplicacao: 128.891 ms | processo: 207.790 ms
Bloco 049/100 | tentativas: 42 | aplicacao: 104.692 ms | processo: 205.076 ms
Bloco 050/100 | tentativas: 235 | aplicacao: 112.066 ms | processo: 180.640 ms
Bloco 051/100 | tentativas: 48 | aplicacao: 105.960 ms | processo: 172.978 ms
Bloco 052/100 | tentativas: 294 | aplicacao: 111.191 ms | processo: 179.909 ms
Bloco 053/100 | tentativas: 35 | aplicacao: 139.348 ms | processo: 208.196 ms
Bloco 054/100 | tentativas: 64 | aplicacao: 133.436 ms | processo: 204.917 ms
```

Figura 27: Exercício 4.1: primeira parte dos registos da experiência de 100 blocos, com alvo 1024.

```
Bloco 054/100 | tentativas: 64 | aplicacao: 133.436 ms | processo: 204.917 ms
Bloco 055/100 | tentativas: 17 | aplicacao: 160.159 ms | processo: 230.217 ms
Bloco 056/100 | tentativas: 45 | aplicacao: 108.756 ms | processo: 174.234 ms
Bloco 057/100 | tentativas: 68 | aplicacao: 116.652 ms | processo: 177.721 ms
Bloco 058/100 | tentativas: 24 | aplicacao: 130.362 ms | processo: 203.088 ms
Bloco 059/100 | tentativas: 3 | aplicacao: 146.399 ms | processo: 220.427 ms
Bloco 060/100 | tentativas: 70 | aplicacao: 133.984 ms | processo: 237.342 ms
Bloco 061/100 | tentativas: 24 | aplicacao: 119.535 ms | processo: 186.228 ms
Bloco 062/100 | tentativas: 30 | aplicacao: 123.207 ms | processo: 184.544 ms
Bloco 063/100 | tentativas: 333 | aplicacao: 129.848 ms | processo: 191.684 ms
Bloco 064/100 | tentativas: 47 | aplicacao: 149.863 ms | processo: 232.617 ms
Bloco 065/100 | tentativas: 43 | aplicacao: 138.382 ms | processo: 215.255 ms
Bloco 066/100 | tentativas: 56 | aplicacao: 132.706 ms | processo: 193.122 ms
Bloco 067/100 | tentativas: 12 | aplicacao: 139.095 ms | processo: 200.381 ms
Bloco 068/100 | tentativas: 11 | aplicacao: 130.340 ms | processo: 194.978 ms
Bloco 069/100 | tentativas: 201 | aplicacao: 171.438 ms | processo: 246.700 ms
Bloco 070/100 | tentativas: 17 | aplicacao: 203.039 ms | processo: 282.578 ms
Bloco 071/100 | tentativas: 44 | aplicacao: 147.533 ms | processo: 222.206 ms
Bloco 072/100 | tentativas: 39 | aplicacao: 166.041 ms | processo: 249.071 ms
Bloco 073/100 | tentativas: 15 | aplicacao: 167.274 ms | processo: 249.679 ms
Bloco 074/100 | tentativas: 138 | aplicacao: 222.803 ms | processo: 317.400 ms
Bloco 075/100 | tentativas: 50 | aplicacao: 161.295 ms | processo: 228.152 ms
Bloco 076/100 | tentativas: 9 | aplicacao: 182.593 ms | processo: 252.326 ms
Bloco 077/100 | tentativas: 45 | aplicacao: 157.880 ms | processo: 232.563 ms
Bloco 078/100 | tentativas: 119 | aplicacao: 218.278 ms | processo: 299.562 ms
Bloco 079/100 | tentativas: 66 | aplicacao: 196.995 ms | processo: 278.732 ms
Bloco 080/100 | tentativas: 14 | aplicacao: 175.883 ms | processo: 252.983 ms
Bloco 081/100 | tentativas: 46 | aplicacao: 170.721 ms | processo: 250.854 ms
Bloco 082/100 | tentativas: 15 | aplicacao: 163.763 ms | processo: 236.538 ms
Bloco 083/100 | tentativas: 26 | aplicacao: 219.781 ms | processo: 312.313 ms
Bloco 084/100 | tentativas: 169 | aplicacao: 214.665 ms | processo: 308.212 ms
Bloco 085/100 | tentativas: 279 | aplicacao: 216.980 ms | processo: 298.815 ms
Bloco 086/100 | tentativas: 81 | aplicacao: 272.413 ms | processo: 350.403 ms
Bloco 087/100 | tentativas: 80 | aplicacao: 211.203 ms | processo: 295.944 ms
Bloco 088/100 | tentativas: 14 | aplicacao: 192.820 ms | processo: 278.662 ms
Bloco 089/100 | tentativas: 74 | aplicacao: 179.092 ms | processo: 260.275 ms
Bloco 090/100 | tentativas: 22 | aplicacao: 184.463 ms | processo: 264.860 ms
Bloco 091/100 | tentativas: 13 | aplicacao: 190.938 ms | processo: 265.433 ms
Bloco 092/100 | tentativas: 18 | aplicacao: 157.080 ms | processo: 228.396 ms
Bloco 093/100 | tentativas: 3 | aplicacao: 159.072 ms | processo: 222.324 ms
Bloco 094/100 | tentativas: 16 | aplicacao: 172.674 ms | processo: 245.645 ms
Bloco 095/100 | tentativas: 15 | aplicacao: 255.275 ms | processo: 332.752 ms
Bloco 096/100 | tentativas: 30 | aplicacao: 190.401 ms | processo: 268.671 ms
Bloco 097/100 | tentativas: 183 | aplicacao: 172.507 ms | processo: 238.239 ms
Bloco 098/100 | tentativas: 115 | aplicacao: 191.714 ms | processo: 268.264 ms
Bloco 099/100 | tentativas: 66 | aplicacao: 219.614 ms | processo: 301.718 ms
Bloco 100/100 | tentativas: 3 | aplicacao: 240.315 ms | processo: 330.562 ms

Experiencia 4.1
Blocos inseridos: 100
Alvo: 1024
Java: 21.0.12.1
Sistema: Linux (amd64)
Total de tentativas: 6771
Media de tentativas por bloco: 67.710
Tempo medio da aplicacao: 127.754 ms
Tempo medio do processo: 207.469 ms
Aplicacao: de main ate a escrita dos ficheiros, sem arranque da JVM.
Processo: inclui lancamento da JVM, aplicacao e escrita da saida no log.
Tabela completa: /mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/blockchain_ex4/resultados_4_1
/execucao_20261022_175904_156/resultados.csv
```

Figura 28: Exercício 4.1: registos finais e médias obtidas pelo grupo com alvo 1024.

Considerando uma distribuição aproximadamente uniforme dos 65536 valores possíveis de H16, a probabilidade de sucesso por tentativa é $1024/65536 = 1/64$. O número médio teórico de tentativas é, portanto, 64. A média observada, 67,710 tentativas, ficou próxima desse valor; a diferença é compatível com a natureza aleatória do processo e com o número finito de blocos analisados.

O tempo médio do processo é superior ao tempo interno porque inclui o lançamento da JVM e outras operações de gestão do processo. Além disso, o tempo de cada inserção não depende apenas da procura do *nonce*: incorpora também a leitura, a verificação e a escrita dos ficheiros existentes. O relatório experimental assinala uma tendência de aumento dos tempos ao longo da cadeia, compatível com o crescimento do número de blocos verificados antes de cada inserção.

4.2 Experiência com alvo 512: resultados e comparação

A experiência foi repetida com uma cadeia inicialmente vazia e 100 mensagens aleatórias, mantendo o procedimento anterior mas reduzindo o alvo para 512. Foi utilizado o mesmo critério de medição do tempo total, desde o lançamento do programa Java até à sua conclusão.

Tabela 14: Comparação dos resultados experimentais para os alvos 1024 e 512.

Indicador	Alvo 1024	Alvo 512
Blocos inseridos	100	100
Total de tentativas	6771	14731
Média de tentativas por bloco	67,710	147,310
Tempo médio interno da aplicação	127,754 ms	157,724 ms
Tempo médio total do processo	207,469 ms	246,139 ms
Média teórica de tentativas	64	128

```
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/blockchain_ex4$ javac --release 21
 -encoding UTF-8 Blockchain.java Experiencia41.java
gonca@Amigo:/mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/blockchain_ex4$ java Experiencia41
 resultados_4_2/test1
Resultados: /mnt/c/Users/gonca/Documents/Licenciatura/ano_3/SegInf/repo/SegInf/blockchain_ex4/resultados_4_2/test
1
Insercoes: 100 | Alvo: 512
Bloco 001/100 | tentativas: 118 | aplicacao: 86.801 ms | processo: 194.208 ms
Bloco 002/100 | tentativas: 188 | aplicacao: 68.550 ms | processo: 174.935 ms
Bloco 003/100 | tentativas: 7 | aplicacao: 67.204 ms | processo: 170.240 ms
Bloco 004/100 | tentativas: 21 | aplicacao: 53.691 ms | processo: 158.453 ms
Bloco 005/100 | tentativas: 76 | aplicacao: 53.741 ms | processo: 121.268 ms
Bloco 006/100 | tentativas: 55 | aplicacao: 52.578 ms | processo: 125.283 ms
Bloco 007/100 | tentativas: 626 | aplicacao: 73.366 ms | processo: 171.479 ms
Bloco 008/100 | tentativas: 127 | aplicacao: 122.706 ms | processo: 210.474 ms
Bloco 009/100 | tentativas: 28 | aplicacao: 62.634 ms | processo: 126.291 ms
Bloco 010/100 | tentativas: 409 | aplicacao: 81.377 ms | processo: 175.452 ms
Bloco 011/100 | tentativas: 160 | aplicacao: 62.442 ms | processo: 129.916 ms
Bloco 012/100 | tentativas: 104 | aplicacao: 67.710 ms | processo: 141.631 ms
Bloco 013/100 | tentativas: 263 | aplicacao: 75.260 ms | processo: 151.886 ms
Bloco 014/100 | tentativas: 16 | aplicacao: 78.458 ms | processo: 212.299 ms
Bloco 015/100 | tentativas: 204 | aplicacao: 70.897 ms | processo: 149.269 ms
Bloco 016/100 | tentativas: 378 | aplicacao: 71.775 ms | processo: 158.823 ms
Bloco 017/100 | tentativas: 369 | aplicacao: 94.574 ms | processo: 174.563 ms
Bloco 018/100 | tentativas: 146 | aplicacao: 74.452 ms | processo: 141.542 ms
Bloco 019/100 | tentativas: 126 | aplicacao: 75.425 ms | processo: 149.136 ms
Bloco 020/100 | tentativas: 813 | aplicacao: 79.787 ms | processo: 191.887 ms
Bloco 021/100 | tentativas: 260 | aplicacao: 83.761 ms | processo: 158.527 ms
Bloco 022/100 | tentativas: 50 | aplicacao: 77.819 ms | processo: 149.575 ms
Bloco 023/100 | tentativas: 31 | aplicacao: 78.602 ms | processo: 156.533 ms
Bloco 024/100 | tentativas: 224 | aplicacao: 104.649 ms | processo: 182.725 ms
Bloco 025/100 | tentativas: 47 | aplicacao: 84.199 ms | processo: 211.965 ms
Bloco 026/100 | tentativas: 73 | aplicacao: 140.778 ms | processo: 239.246 ms
Bloco 027/100 | tentativas: 175 | aplicacao: 83.013 ms | processo: 162.136 ms
Bloco 028/100 | tentativas: 198 | aplicacao: 93.582 ms | processo: 181.674 ms
Bloco 029/100 | tentativas: 102 | aplicacao: 84.841 ms | processo: 157.022 ms
Bloco 030/100 | tentativas: 186 | aplicacao: 108.556 ms | processo: 185.674 ms
Bloco 031/100 | tentativas: 259 | aplicacao: 129.233 ms | processo: 195.627 ms
Bloco 032/100 | tentativas: 44 | aplicacao: 90.840 ms | processo: 160.656 ms
Bloco 033/100 | tentativas: 63 | aplicacao: 88.780 ms | processo: 167.121 ms
Bloco 034/100 | tentativas: 82 | aplicacao: 110.914 ms | processo: 180.597 ms
Bloco 035/100 | tentativas: 58 | aplicacao: 91.997 ms | processo: 157.700 ms
Bloco 036/100 | tentativas: 78 | aplicacao: 108.177 ms | processo: 249.065 ms
Bloco 037/100 | tentativas: 131 | aplicacao: 95.005 ms | processo: 167.972 ms
Bloco 038/100 | tentativas: 12 | aplicacao: 109.001 ms | processo: 185.853 ms
Bloco 039/100 | tentativas: 72 | aplicacao: 116.321 ms | processo: 189.379 ms
Bloco 040/100 | tentativas: 6 | aplicacao: 137.326 ms | processo: 215.781 ms
Bloco 041/100 | tentativas: 23 | aplicacao: 103.120 ms | processo: 235.295 ms
Bloco 042/100 | tentativas: 387 | aplicacao: 117.774 ms | processo: 192.533 ms
Bloco 043/100 | tentativas: 56 | aplicacao: 107.146 ms | processo: 192.067 ms
Bloco 044/100 | tentativas: 223 | aplicacao: 152.320 ms | processo: 238.126 ms
Bloco 045/100 | tentativas: 309 | aplicacao: 174.153 ms | processo: 248.545 ms
Bloco 046/100 | tentativas: 3 | aplicacao: 163.298 ms | processo: 258.451 ms
Bloco 047/100 | tentativas: 274 | aplicacao: 127.867 ms | processo: 212.351 ms
Bloco 048/100 | tentativas: 88 | aplicacao: 127.991 ms | processo: 193.582 ms
Bloco 049/100 | tentativas: 472 | aplicacao: 106.202 ms | processo: 173.740 ms
Bloco 050/100 | tentativas: 43 | aplicacao: 167.231 ms | processo: 237.592 ms
```

Figura 29: Exercício 4.2: primeira parte da experiência de 100 blocos com alvo 512.

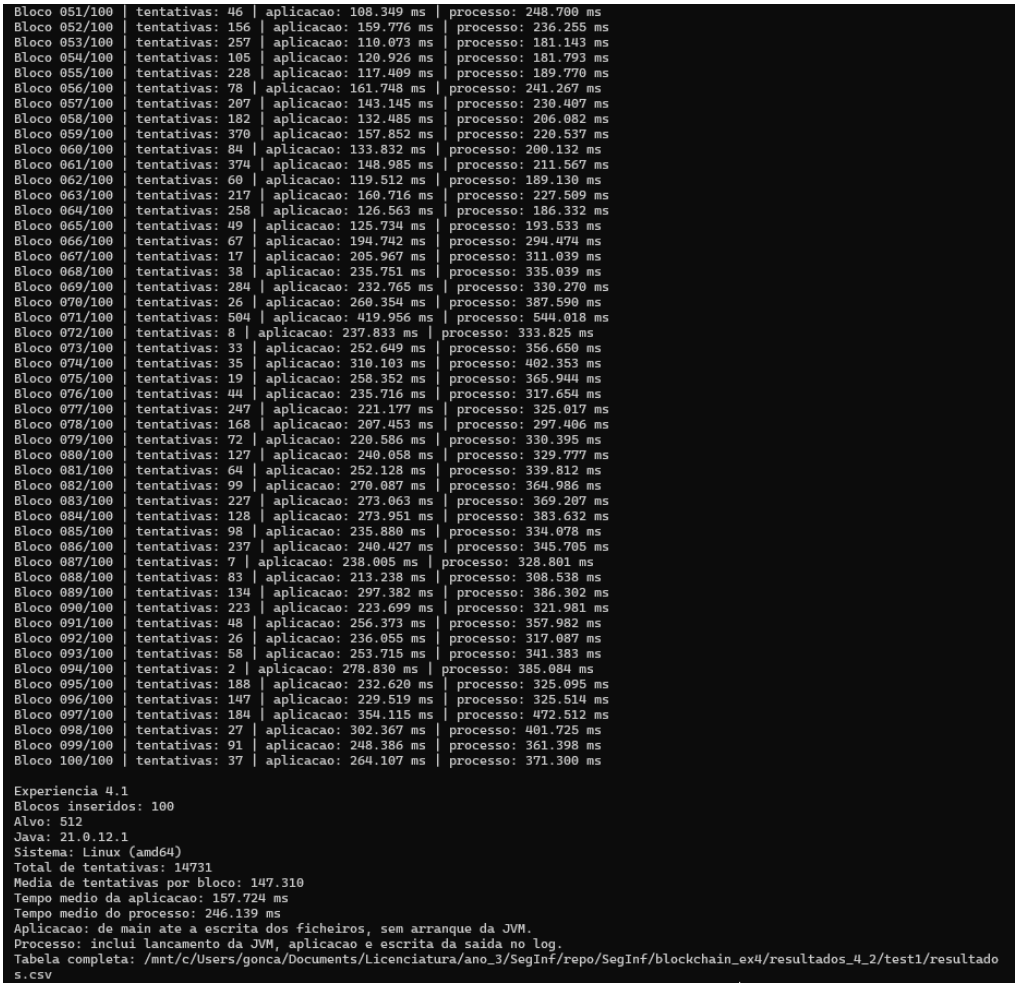


Figura 30: Exercício 4.2: registos finais e médias da experiência com alvo 512.

Ao passar de 1024 para 512, o conjunto de valores H16 admissíveis é reduzido para metade: anteriormente eram aceites os valores de 0 a 1023 e, com o novo limite, apenas os valores de 0 a 511. A probabilidade teórica de sucesso por tentativa diminui de 1/64 para 1/128, duplicando o número médio teórico de tentativas.

Na experiência, a média aumentou de 67,710 para 147,310 tentativas por bloco, ou seja, aproximadamente 2,18 vezes. O tempo médio total de cada processo aumentou de 207,469 ms para 246,139 ms, correspondendo a aproximadamente 18,6%. Os tempos não duplicaram porque incluem custos de arranque da JVM, verificação da cadeia e leitura e escrita de ficheiros, para além da pesquisa do *nonce*. Os resultados mostram que a redução do alvo aumenta a dificuldade da prova de trabalho, ainda que o tempo total não seja diretamente proporcional ao número de tentativas.

4.3 Alteração do quinto bloco e estimativa do trabalho necessário

Na última alínea, considerou-se o cenário de um atacante que pretende modificar a mensagem do quinto bloco de uma cadeia com 100 blocos, sem que a cadeia deixe de satisfazer as condições de ligação e de prova de trabalho.

Alterar apenas o ficheiro `5.txt` tornaria inconsistente o H16 do bloco 5. Para manter a cadeia coerente, seria necessário procurar um novo *nonce* para o bloco 5, usando o H16 do bloco 4, e atualizar os valores correspondentes em `5.nce` e `5.hsh`. Como o bloco 6 depende do H16 do bloco 5, seria depois necessário recalcular o *nonce* e o H16 do bloco 6, repetindo sucessivamente o processo até ao bloco 100. As mensagens dos blocos seguintes poderiam manter-se, mas os respetivos *nonces* e hashes teriam de ser recalculados. Os blocos 1 a 4 permaneceriam inalterados.

Tabela 15: Estimativa da reconstrução da cadeia após alteração do quinto bloco.

Parâmetro	**Valor ou procedimento**
Primeiro bloco afetado	5
Último bloco afetado	100
Número de blocos a reconstruir	$100 - 5 + 1 = 96$
Valor alvo considerado	512
Tempo médio por processo da alínea anterior	246,139 ms
Estimativa total	$96 \times 246,139\,\text{ms} = 23\,629,344\,\text{ms}$
Equivalente em segundos	$\approx 23,63$ s

A estimativa de 23,63 segundos corresponde à repetição de 96 inserções independentes, utilizando o tempo médio de processo observado na experiência com alvo 512. Trata-se de uma aproximação obtida a partir das medições do grupo, e não de uma medição direta de um ataque executado. O valor real dependeria do número de tentativas de cada bloco e das condições de execução; uma implementação que reconstruísse os blocos num único processo poderia apresentar tempos diferentes.

Conclusão do exercício 4

A experiência evidenciou a dependência entre blocos proporcionada pelos valores H16 e demonstrou que a redução do alvo aumenta a dificuldade da prova de trabalho. Os resultados observados ficaram próximos da tendência prevista pelo modelo probabilístico, embora influenciados pelo arranque da JVM e pela gestão dos ficheiros. A alteração de um bloco intermédio exige a reconstrução dos blocos seguintes para manter a consistência da cadeia; o custo estimado para o cenário estudado foi de cerca de 23,63 segundos. Trata-se de um modelo didático com hashes truncados a 16 bits, e não de uma análise de segurança de uma blockchain real.

Exercício 5 – Proteção de texto com JWE e a biblioteca JCA

Este exercício teve como objetivo desenvolver uma aplicação de consola para proteger mensagens de texto através da *Java Cryptography Architecture* (JCA), representando o resultado no formato *JSON Web Encryption* (JWE). A implementação segue o princípio da criptografia híbrida: combina a cifra autenticada AES-GCM com a proteção da chave simétrica através de RSA-OAEP, de acordo com a abordagem descrita na secção 3.3 da RFC 7516 [12].

O trabalho foi desenvolvido em quatro etapas, correspondentes às alíneas do enunciado: cifra e decifra AES-GCM; proteção da chave AES com RSA; obtenção de chaves a partir de certificados e keystores; e integração dos componentes numa aplicação que permite produzir e interpretar tokens JWE. Nesta secção apresentam-se os objetivos, as decisões técnicas, a descrição das operações e os resultados obtidos. **O código-fonte Java e os comentários de implementação são disponibilizados em separado**, sem reprodução integral no corpo ou no fim deste relatório.

5.1 Alínea a) – Cifra e decifra autenticada AES-GCM

O primeiro componente foi desenvolvido para cifrar e decifrar mensagens de texto, recebendo a mensagem, os dados adicionais autenticados, a chave AES, o vetor de inicialização (IV) e o comprimento da marca de autenticação. O enunciado exige também que o criptograma possa ser distinguido da respetiva *tag*.

Na implementação, a classe `cifra` utiliza a transformação `AES/GCM/NoPadding` da JCA. O objeto `GCMParameterSpec` reúne os parâmetros do modo GCM, nomeadamente o IV e o tamanho da marca em bits. Após a inicialização para cifra, os dados adicionais são autenticados antes de se processar a mensagem, convertida numa sequência de bytes em UTF-8. A operação criptográfica devolve a concatenação do criptograma com a marca de autenticação.

Para cumprir o requisito de separação, a implementação calcula o comprimento da marca em bytes e extrai os dois elementos. O tipo `Resultado`, implementado através de um `record` Java, permite devolver ambos separadamente. No processo inverso, os elementos são novamente concatenados na ordem exigida pelo AES-GCM; a decifra só devolve o texto se a verificação da marca for bem-sucedida.

A geração dos parâmetros de teste foi efetuada com `KeyGenerator` para a chave AES e `SecureRandom` para o IV. O IV tem 12 bytes; a marca tem 16 bytes. A configuração A256GCM utilizada na aplicação JWE corresponde a uma CEK de 256 bits. Um IV não deve voltar a ser utilizado com a mesma chave AES, porque a sua reutilização compromete a segurança do modo GCM.

Resultados do teste. No terminal, a aplicação apresentou o criptograma e a marca codificados em Base64 e recuperou a mensagem original: *«Este trabalho pertence ao grupo 3 da turma 51D»*. Num teste posterior, a alteração intencional do criptograma provocou uma falha de autenticação, sem recuperação normal da mensagem. O resultado demonstra o funcionamento da cifra e da

decifra e a deteção da alteração realizada.

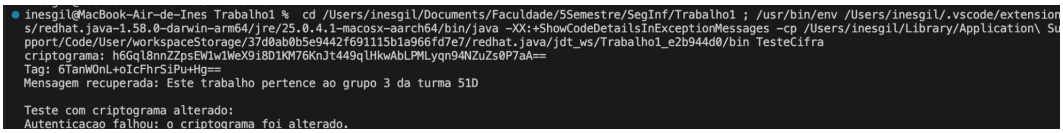


Figura 31: Exercício 5(a): recuperação da mensagem AES-GCM e rejeição de um criptograma intencionalmente alterado.

Interpretação

O teste confirma a recuperação da mensagem original quando a chave, o IV e os AAD coincidem. Quando o criptograma é modificado, a marca deixa de corresponder aos dados e a autenticação falha. Durante o desenvolvimento verificou-se também que a cifra e a decifra devem empregar a mesma codificação de caracteres nos AAD; no caso concreto, a versão final testada foi corrigida para UTF-8 em ambas as operações.

5.2 Alínea b) – Proteção da chave AES com RSA-OAEP

O segundo componente teve como objetivo cifrar e decifrar uma chave AES utilizando RSA. Esta operação não cifra diretamente o texto da mensagem: protege a chave simétrica que será necessária à decifra do conteúdo.

A implementação da classe `chaves` utiliza a classe `Cipher`, configurada para RSA com OAEP. Os parâmetros de OAEP são definidos explicitamente através de `OAEPParameterSpec`, selecionando SHA-1 e MGF1 com SHA-1, conforme o identificador `RSA-OAEP` do JWE. Durante a cifra, obtêm-se os bytes da chave AES e processam-se com a chave pública RSA do destinatário. Na decifra, a chave privada correspondente recupera os bytes, a partir dos quais é reconstruído um objeto `SecretKey` para AES.

Para o teste isolado, a aplicação gerou uma chave AES de 256 bits e um par de chaves RSA de 2048 bits. A chave AES foi cifrada e posteriormente recuperada, sendo comparados os bytes originais e recuperados através do método `Arrays.equals`.

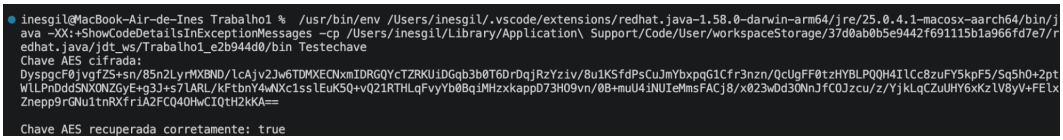


Figura 32: Exercício 5(b): chave AES cifrada com RSA-OAEP e comparação positiva após a recuperação.

Interpretação

A mensagem `Chave AES recuperada corretamente: true` confirma que os bytes da chave recuperada são iguais aos da chave inicialmente gerada. A apresentação da chave cifrada em Base64 facilita apenas a visualização do resultado binário; o Base64 não acrescenta proteção

criptográfica.

5.3 Alínea c) – Certificados X.509 e keystores PKCS#12

A terceira etapa substituiu o par RSA gerado exclusivamente para os testes por chaves associadas a um destinatário concreto. O enunciado exige a obtenção da chave pública de um certificado X.509 previamente validado e a recuperação da chave privada a partir de um keystore.

Validação da cadeia de certificação. O teste utilizou o certificado final `Alice_1.cer`, o certificado da autoridade intermédia `CA1-int.cer` e o certificado raiz `CA1.cer`. Os certificados foram lidos através de `CertificateFactory` e representados como objetos `X509Certificate`. Foi construído um caminho com Alice e a entidade intermédia, enquanto a CA raiz foi configurada separadamente como *trust anchor*. A validação foi efetuada com o mecanismo PKIX disponibilizado por `CertPathValidator`. A chave pública só foi devolvida depois de a validação terminar com sucesso.

Por não terem sido configurados mecanismos de consulta do estado de revogação, a validação foi realizada com a verificação de revogação desativada. Esta limitação é relevante: a validação obtida não prova que os certificados não tenham sido entretanto revogados.

```
inesgil@MacBook-Air-de-Ines Trabalho1 %  /usr/bin/env /Users/inesgil/.vscode/extensions/redhat.java-1.58.0-darwin-arm64/jre/25.0.4.1-macosx-aarch64/bin/j
ava -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/inesgil/Library/Application\ Support/Code/User/workspaceStorage/37d0ab0b5e9442f691115b1a966fd7e7/r
edhat.java/jdt_ws/Trabalho1_e2b944d0/bin Testecertificados
Certificado validado com sucesso!
Algoritmo da chave publica: RSA
```

Figura 33: Exercício 5(c): validação do certificado de Alice e identificação da sua chave pública RSA.

Recuperação da chave privada. Para a chave privada foi utilizado o ficheiro `Alice_1.pfx`, no formato PKCS#12. A classe `FicheiroPFX` recorre a `KeyStore` para carregar o conteúdo, percorrer os *aliases*, identificar uma entrada de chave e recuperar um objeto `PrivateKey`. O programa solicita a palavra-passe no momento da execução; a chave privada não deve ser apresentada nem copiada para o relatório.

Teste de integração das duas chaves. Foi gerada uma CEK AES e cifrada com a chave pública obtida do certificado validado de Alice. Depois, a CEK foi recuperada com a chave privada proveniente do PFX correspondente. O resultado `Chaves AES iguais: true` demonstra que as duas chaves RSA podem ser utilizadas como par nas operações de RSA-OAEP realizadas.

```
inesgil@MacBook-Air-de-Ines Trabalho1 %  /usr/bin/env /Users/inesgil/.vscode/extensions/redhat.java-1.58.0-darwin-arm64/jre/25.0.4.1-macosx-aarch64/bin/j
ava -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/inesgil/Library/Application\ Support/Code/User/workspaceStorage/37d0ab0b5e9442f691115b1a966fd7e7/r
edhat.java/jdt_ws/Trabalho1_e2b944d0/bin TesteAliceChave
Certificado de Alice validado com sucesso!
Palavra-passe do PFX
Chave privada recuperaga com sucesso!
Chave AES cifrada com a chave publica de Alice.
Chaves AES iguais: true
inesgil@MacBook-Air-de-Ines Trabalho1 %
```

Figura 34: Exercício 5(c): validação do certificado, recuperação da chave privada do PFX e confirmação da igualdade da chave AES.

Interpretação

O resultado mostra a associação funcional entre o certificado X.509 de Alice e o respetivo keystore: a chave pública permite proteger a CEK e a chave privada correspondente permite recuperá-la. O teste foi efetuado com a cadeia CA1 e não equivale a uma validação automática de quaisquer outros destinatários ou hierarquias de certificação.

5.4 Alínea d) – Integração na aplicação JSON Web Encryption

Na etapa final, os componentes anteriores foram integrados na classe `JWE`. A aplicação disponibiliza a operação `enc`, que recebe uma mensagem e o certificado do destinatário, e a operação `dec`, que recebe um token e o ficheiro PFX do destinatário. A palavra-passe é solicitada durante a segunda operação, permitindo aceder à chave privada sem a colocar como argumento do comando.

Estrutura do token. A serialização compacta JWE contém cinco componentes codificados em Base64URL, separados por pontos e apresentados sempre na mesma ordem:

Cabeçalho protegido · **Chave cifrada** · **IV** · **Criptograma** · **Tag**

O cabeçalho protegido identifica os algoritmos `RSA-OAEP` e `A256GCM`. O segundo componente contém a CEK cifrada com a chave pública RSA. O terceiro contém o IV aleatório de 96 bits. O quarto guarda o criptograma AES-GCM e o quinto a marca de autenticação de 128 bits.

Tabela 16: Elementos da serialização compacta JWE utilizada na aplicação.

Elemento	Função
Cabeçalho protegido	Identifica os algoritmos usados e integra os AAD autenticados pelo AES-GCM.
Chave cifrada	CEK protegida com a chave pública RSA do destinatário.
IV	Vetor de inicialização necessário à cifra e à decifra AES-GCM.
Criptograma	Conteúdo da mensagem protegido por cifra simétrica.
Tag	Marca de autenticação utilizada para verificar os dados protegidos.

Construção do token. O processo começa com a validação do certificado do destinatário e a obtenção da respetiva chave pública. É então gerada uma CEK AES de 256 bits, protegida com RSA-OAEP. Seguidamente, cria-se o cabeçalho JWE e a sua representação Base64URL. A cifra AES-GCM recebe a mensagem, a CEK, o IV aleatório e, como AAD, o cabeçalho já codificado. Finalmente, são codificados os elementos binários e construído o token com as cinco partes exigidas pela RFC 7516.

Interpretação e decifra do token. Na operação inversa, a aplicação separa as cinco partes, descodifica os seus conteúdos e verifica a configuração de algoritmos suportada. Depois, recupera

a chave privada do PFX e aplica RSA-OAEP para recuperar a CEK. Por fim, a CEK e o IV são utilizados na decifra AES-GCM, com o cabeçalho protegido como AAD. A mensagem só é devolvida quando a verificação da tag termina com sucesso.

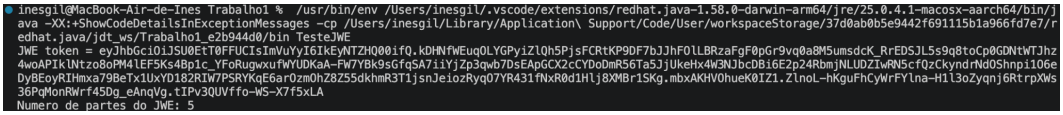


Figura 35: Exercício 5(d): token JWE gerado e confirmação da existência das cinco partes previstas na serialização compacta.

Interpretação

A apresentação `Numero de partes do JWE: 5` confirma a estrutura textual criada. Esta contagem, por si só, não demonstra o funcionamento criptográfico: para essa verificação foi necessário realizar também a operação de decifra com a chave privada do destinatário.

Resultados da cifra e decifra completas. A aplicação foi executada na linha de comandos, primeiro para gerar um token a partir da mensagem *«Este trabalho pertence ao grupo 3 da turma 51D»* e do certificado de Alice e, depois, para recuperar a mesma mensagem com o respetivo PFX. A operação de decifra apresentou `Decrypted text = Este trabalho pertence ao grupo 3 da turma 51D`. Num teste integrado adicional foi obtido `Mensagens iguais: true`, confirmando a correspondência entre o texto original e o texto recuperado.



Figura 36: Exercício 5(d): execução das operações de consola `enc` e `dec`, com recuperação da mensagem original.

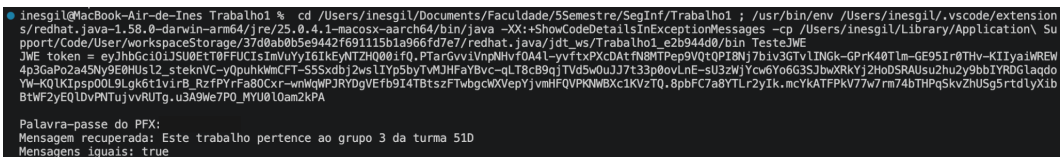


Figura 37: Exercício 5(d): teste integrado de criação e decifra do JWE, com comparação positiva das mensagens.

Teste de segurança: alteração intencional do criptograma

Para testar a verificação de integridade, foi criado um JWE válido e realizada a sua decifra com sucesso. Em seguida, o token foi separado nas cinco partes, a quarta parte foi descodificada e

alterou-se um bit do primeiro byte do criptograma. Depois de o voltar a codificar em Base64URL, reconstruiu-se o token mantendo inalterados o cabeçalho, a chave cifrada, o IV e a tag original.

Na segunda tentativa de decifra, a tag deixou de corresponder ao criptograma modificado. A implementação AES-GCM rejeitou a operação, originando uma falha de autenticação. No terminal foi apresentada a mensagem `Autenticacao falhou!`, acompanhada da indicação de que o AES-GCM detetou a alteração do criptograma.

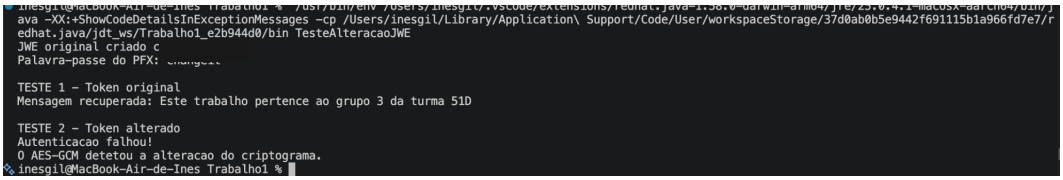


Figura 38: Exercício 5(d): mensagem recuperada do token original e rejeição de um token com criptograma alterado.

Interpretação

O teste demonstra que uma modificação intencional do criptograma é detetada pela verificação da tag GCM. Esta conclusão diz respeito ao cenário efetivamente executado; não se devem apresentar testes com outros destinatários, cabeçalhos ou implementações independentes como concluídos sem os respetivos resultados.

Limitações e considerações de segurança

A implementação foi testada com o material criptográfico de Alice e com a cadeia de certificação CA1 configurada para esse destinatário. A verificação de revogação foi desativada; numa aplicação de produção seria necessário configurá-la adequadamente. O cabeçalho JWE é reconhecido pela representação JSON esperada no programa, o que limita a interoperabilidade com cabeçalhos semanticamente equivalentes, mas serializados de outra forma. Os ensaios documentados confirmam a criação e a decifra do token e a rejeição de um criptograma alterado, mas não comprovam a compatibilidade com implementações JWE independentes.

Outra limitação decorre do mecanismo de transporte de chaves por RSA: se a chave privada do destinatário for comprometida posteriormente, tokens antigos que tenham sido guardados poderão também ser decifrados. Por isso, esta construção não oferece *perfect forward secrecy*.

Conclusão do exercício 5

Foram integrados os componentes de cifra autenticada AES-GCM, de proteção da CEK com RSA-OAEP, de validação de certificados X.509 e de recuperação de chaves privadas a partir de keystores PKCS#12. A aplicação permitiu gerar um token JWE compacto e recuperar corretamente a mensagem original através de operações de consola. A comparação positiva entre as mensagens e a rejeição de um token intencionalmente alterado constituem evidência do funcionamento e da autenticação nos testes realizados. A descrição técnica e os prints permanecem neste

relatório; o código Java comentado é disponibilizado como material separado.

Conclusão

A realização deste primeiro trabalho de Segurança Informática permitiu aplicar, de forma prática, conceitos fundamentais de criptografia e de confiança digital. As diferentes atividades, realizadas com OpenSSL e com a *Java Cryptography Architecture* (JCA), demonstraram como mecanismos criptográficos com objetivos distintos podem ser utilizados isoladamente ou combinados para proteger informação e verificar a sua autenticidade.

No primeiro exercício, a proteção de ficheiros com AES-128-CBC e HMAC-SHA256 permitiu distinguir o papel da cifra simétrica, responsável pela confidencialidade, do papel da marca de autenticação, utilizada para detetar alterações. A análise do *padding* e a reconstrução do ficheiro original ajudaram a compreender a necessidade de preservar os parâmetros da cifra e de verificar a autenticidade dos dados recebidos antes de os decifrar.

No segundo exercício, foram utilizadas assinaturas digitais RSA com SHA-256 para verificar mensagens e ficheiros. A assinatura de imagens de diferentes dimensões, a alteração intencional de uma mensagem e a experiência com as chaves de Mallory demonstraram que a validade de uma assinatura depende simultaneamente da integridade dos dados e da utilização da chave pública correspondente ao signatário. Assim, os ensaios permitiram distinguir as garantias de integridade e autenticidade oferecidas pelas assinaturas digitais da confidencialidade proporcionada pela cifra.

O terceiro exercício permitiu construir e analisar uma hierarquia de certificação composta por uma autoridade raiz, uma autoridade intermédia e um certificado de servidor. A configuração das extensões X.509, o cálculo de impressões digitais e os testes de validação mostraram a importância de uma cadeia de confiança correta e da correspondência do nome do servidor com a identidade inscrita no certificado. Foi igualmente demonstrado que o algoritmo usado para assinar um certificado depende da chave da entidade emissora, não tendo de coincidir com o algoritmo da chave pública do seu titular.

Na segunda parte do trabalho, o exercício da cadeia de blocos mostrou a ligação entre registos através de hashes e uma forma simplificada de prova de trabalho. Os resultados obtidos com 100 blocos indicaram que a redução do valor alvo de 1024 para 512 aumentou a média de tentativas de 67,710 para 147,310. A análise da alteração do quinto bloco evidenciou que uma modificação intermédia obriga ao recálculo dos blocos seguintes, tendo sido estimados aproximadamente 23,63 segundos para reconstruir a cadeia até ao bloco 100, nas condições consideradas. Estas conclusões dizem respeito ao modelo experimental desenvolvido, com H16 de apenas 16 bits.

Finalmente, no quinto exercício, foram integrados os mecanismos de AES-256-GCM, RSA-OAEP, validação de certificados X.509 e acesso a chaves privadas em keystores PKCS#12. A aplicação Java permitiu produzir tokens *JSON Web Encryption* (JWE) e recuperar a mensagem original através das operações `enc` e `dec`. O teste de alteração intencional do criptograma confirmou que a verificação da marca AES-GCM rejeita dados modificados. Este último exercício reuniu, numa mesma aplicação, os conceitos de confidencialidade, autenticação e proteção de chaves estudados anteriormente.

Em síntese, os resultados documentados cumpriram os objetivos experimentais principais do trabalho e reforçaram a importância da correta escolha de algoritmos, parâmetros, chaves e

mecanismos de validação. Importa, contudo, distinguir as implementações didáticas de soluções prontas para produção: a aplicação JWE foi testada com uma cadeia de certificação específica e com a revogação desativada, enquanto a cadeia de blocos utiliza uma função H16 reduzida. O trabalho permitiu, deste modo, consolidar conhecimentos teóricos através da implementação, da observação de resultados e da análise das limitações dos mecanismos utilizados.

Referências

[1] ISEL, *Segurança Informática – Primeiro trabalho, Semestre de Inverno de 2026/27*, enunciado, 2026.

[2] *Funções de hash*, slides 03 da unidade curricular de Segurança Informática, ISEL, 2026.

[3] *Esquemas assimétricos*, slides 04 da unidade curricular de Segurança Informática, ISEL, 2026.

[4] Repositório da unidade curricular de Segurança Informática, pasta `OpenSSL` (`openssl-asymmetric-example-commands.md`), https://github.com/isel-deetc-computersecurity/seginf-public/.

[5] Grupo 03, *SegInf: repositório dos trabalhos de grupo de Segurança Informática*, https://github.com/Amigo056/SegInf, consultado em outubro de 2026.

[6] *OpenSSL Docs*, https://docs.openssl.org/master/man1/.

[7] *Comando cat*, https://man7.org/linux/man-pages/man1/cat.1.html.

[8] *Comando gc (PowerShell)*, https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.management/get-content.

[9] *OpenSSL command enc*, https://docs.openssl.org/master/man1/openssl-enc/.

[10] *OpenSSL command mac*, https://docs.openssl.org/master/man1/openssl-mac/.

[11] Oracle, *Java Cryptography Architecture (JCA) Reference Guide*, https://docs.oracle.com/en/java/javase/21/security/java-cryptography-architecture-jca-reference-guide.html.

[12] M. Jones, J. Hildebrand, *JSON Web Encryption (JWE)*, RFC 7516, https://datatracker.ietf.org/doc/html/rfc7516.

[13] *Infraestruturas de chave pública*, slides 06 da unidade curricular de Segurança Informática, ISEL, 2026.

[14] *Transport Layer Security*, slides 07 da unidade curricular de Segurança Informática, ISEL, 2026.

[15] Notas das aulas 6 a 10 da unidade curricular, repositório `seginf-public`, pasta `Lecture-Notes`, https://github.com/isel-deetc-computersecurity/seginf-public/tree/main/Lecture-Notes.

[16] Repositório da unidade curricular, *Validador online*, https://github.com/isel-deetc-computersecurity/seginf-public/blob/main/validador-online.md.

[17] Oracle, *Classe java.util.Base64*, https://docs.oracle.com/javase/8/docs/api/java/util/Base64.html.

[18] M. Jones, *JSON Web Algorithms (JWA)*, RFC 7518, https://datatracker.ietf.org/doc/html/rfc7518.